

Robust Recommender System: A Survey and Future Directions

KAIKE ZHANG, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

QI CAO, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China

FEI SUN, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China

YUNFAN WU, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China

SHUCHANG TAO, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China

HUAWEI SHEN, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

XUEQI CHENG, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China

With the rapid growth of information, recommender systems have become integral for providing personalized suggestions and overcoming information overload. However, their practical deployment often encounters “dirty” data, where noise or malicious information can lead to abnormal recommendations. Research on improving robustness of recommender systems against such dirty data has thus gained significant attention. This survey provides a comprehensive review of recent work on robust recommender systems. We first present a taxonomy to organize current techniques for withstanding malicious attacks and natural noise. We then explore state-of-the-art methods in each category, including fraudster detection, adversarial training, certifiable robust training for defending against malicious attacks, and regularization, purification, self-supervised learning for defending against malicious attacks. Additionally, we summarize evaluation metrics and commonly used

This work is funded by the National Key R&D Program of China (2022YFB3103700, 2022YFB3103701), the National Natural Science Foundation of China under Grant Nos. 62472409, U21B2046, 62272125, the Strategic Priority Research Program of the Chinese Academy of Sciences (XDB0680201), and the Beijing Natural Science Foundation (4252023).

Authors' Contact Information: Kaike Zhang, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China; e-mail: kaiketzhang99@gmail.com; Qi Cao (corresponding author), State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China; e-mail: caoqi@ict.ac.cn; Fei Sun, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China; e-mail: sunfei@ict.ac.cn; YunfanWu, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China; e-mail: wuyunfan19b@ict.ac.cn; Shuchang Tao, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China; e-mail: shuchangtao5@gmail.com; Huawei Shen, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China; e-mail: shenhuawei@ict.ac.cn; Xueqi Cheng, State Key Laboratory of AI Safety, Institute of Computing Technology Chinese Academy of Sciences, Beijing, China and University of Chinese Academy of Sciences, Beijing, China; e-mail: cxq@ict.ac.cn.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 0360-0300/2025/09-ART10

<https://doi.org/10.1145/3757057>

datasets for assessing robustness. We discuss robustness across varying recommendation scenarios and its interplay with other properties like accuracy, interpretability, privacy, and fairness. Finally, we delve into open issues and future research directions in this emerging field. Our goal is to provide readers with a comprehensive understanding of robust recommender systems and to identify key pathways for future research and development. To facilitate ongoing exploration, we maintain a continuously updated GitHub repository with related research: <https://github.com/Kaike-Zhang/Robust-Recommender-System>.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Information systems** → **Recommender systems**; • **Security and privacy** → **Formal security models**;

Additional Key Words and Phrases: Recommender system, robustness, attack, noise, defense, denoise

ACM Reference Format:

Kaike Zhang, Qi Cao, Fei Sun, Yunfan Wu, Shuchang Tao, Huawei Shen, and Xueqi Cheng. 2025. Robust Recommender System: A Survey and Future Directions. *ACM Comput. Surv.* 58, 1, Article 10 (September 2025), 38 pages. <https://doi.org/10.1145/3757057>

1 Introduction

In the era of information overload, recommender systems have emerged as powerful tools for providing personalized suggestions across a wide range of applications [27, 34, 47]. From e-commerce platforms like Amazon and Alibaba to streaming services such as YouTube and Netflix, recommender systems have become an important part of the user experience. These systems not only aid users in their decision-making processes but also contribute to revenue growth for businesses. Recommender systems leverage large amounts of data, including past user behavior, demographic information, and contextual data, to generate personalized recommendations for various products, services, and content [62].

Importance and Influence of Robustness in Recommender Systems. Recommender systems heavily rely on data to provide accurate recommendations to users, using past behavior and preferences to predict future actions [62]. Despite their widespread adoption and value in numerous industries, the natural openness of recommender systems means practical implementations often face the challenge of dealing with dirty data [99, 112, 143]. In 2018, the New York Times reported an article about the gray industry derived from YouTube’s fake views.¹ In the same year, the BBC reported that false comments on online review websites could affect £23 billion in British customer spending.² In 2021, Guardian reported that Facebook employees found more than 30 cases of political manipulation involving 25 countries on the platform.³ Noisy or malicious information can lead to inaccuracies in the recommendations provided, resulting in a poor user experience or reduced business value. Consequently, ensuring the robustness of recommender systems is essential, as it measures recommender systems’ ability to provide stable recommendations when the data is partially damaged [43, 50, 93, 94, 169].

In recent years, robustness in recommender systems has become a focal point of research [6, 44, 99, 125, 140, 143, 189]. This trend is evident in the growing number of published papers on the topic, as shown in Figure 1. According to the chart, research interest in this area has increased significantly since 2019. This surge in attention is also reflected in the inclusion of tutorials and workshops on recommender systems’ robustness at various top-tier conferences. For instance, RecSys featured a tutorial on *Adversarial Learning for Recommendation* [7], dedicated to robustness

¹<https://www.nytimes.com/interactive/2018/08/11/technology/youtube-fake-view-sellers.html>

²<https://www.bbc.com/news/technology-43907695>

³<https://www.theguardian.com/technology/2021/apr/12/facebook-loop-hole-state-backed-manipulation>

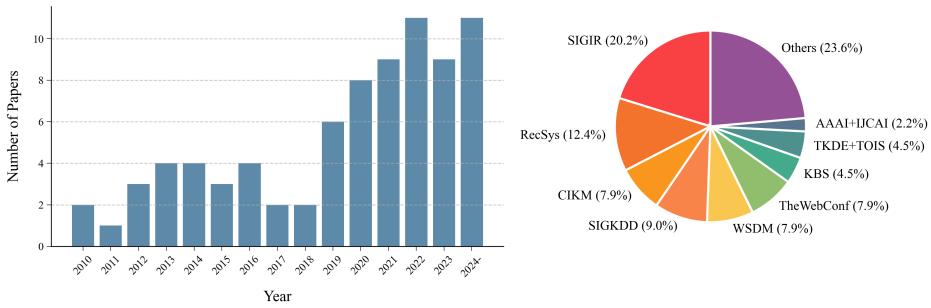


Fig. 1. The statistics of publications related to robust recommender systems with the publication year and conference/journal.

in recommender systems, in 2020. Similarly, TheWebConf hosted a tutorial on *Trustworthy Recommender Systems* in 2023, highlighting robustness as a key topic. Additionally, several workshops have emphasized the importance of robustness in recommender systems. Specifically, SIGIR’s *Causality in Search and Recommendation* (2021) [183], TheWebConf’s *Decision Making for Information Retrieval and Recommender Systems* (2023) [153], and *The 1st Workshop on Human-Centered Recommender Systems* (2025) [178], along with SIGKDD’s *Workshop on Industrial Recommendation Systems* (2021) [154], have identified robustness as a core theme. Therefore, we aim to provide a systematic review of robustness in recommender systems.

Difference with Existing Surveys. Numerous surveys on recommender systems have been published recently. Some focus on specific types of recommender systems, such as graph-based recommender systems [146], knowledge-based recommender systems [123], deep-learning-based recommender systems [179], and reinforcement-learning-based recommender systems [3]. Others examine broader issues beyond accuracy, including trustworthy recommender systems [46], debiased recommender systems [24], explainable recommender systems [182], and evaluation methodologies for recommender systems [5, 168]. To the best of our knowledge, existing surveys related to robustness primarily focus on a limited subset of recommender systems’ robustness, emphasizing adversarial strategies [8, 38] or the detection of malicious users [105]. In contrast, our survey provides a comprehensive perspective on robustness in recommender systems, not only considering both adversarial and malicious behaviors but also exploring techniques for mitigating natural noise. Additionally, while some surveys discuss robustness in Computer Vision and **Natural Language Processing (NLP)** [138, 186], they do not specifically address recommender systems. Given the unique characteristics of recommendation tasks, such as the interdependencies between entities and transductive recommendation scenarios, the strategies for enhancing recommender systems’ robustness may differ significantly from conventional approaches. Therefore, our survey aims to fill this gap by focusing on robustness in recommender systems and exploring the techniques and strategies specifically tailored to recommendation tasks.

Papers Collection. In this survey, we undertook an extensive search across premier conferences and journals, including SIGKDD, SIGIR, Webconf, RecSys, WSDM, CIKM, NeurIPS, ICML, TKDE, TOIS, and so on. We employed search keywords such as “recommender system”, “recommendation”, “collaborative filtering”, and “matrix factorization”, coupled with terms like “robustness”, “denoise”, “defense”, or “detection”, spanning the period from 2010 to 2025. To ensure the representative nature of our paper collection, we further screened the publications. Notably, while a small number of researchers identify *bias* and *sparseness* as issues pertinent to a kind of robustness [107, 185, 189], there is a gap with the robustness in general, which measures recommender systems’ ability to

provide stable recommendations when the data is partially damaged [43, 50, 93, 94, 169]. Therefore, in this survey, we consciously exclude papers that solely focus on issues of bias and sparseness to maintain the specificity and relevance of our review. If the readers are interested in these properties, there are some relevant surveys that have already been conducted on bias [24] and sparseness [2, 49].

Contributions of this survey. We surveyed a collection of papers regarding the robustness of recommender systems to ensure a quick and efficient understanding for researchers entering into this field. Our goal is to further aid the progress in this area. Briefly, the noteworthy contributions of this survey are as follows:

- A comprehensive and systematic taxonomy for robustness-enhancement methods in recommender systems.
- An all-encompassing overview of the representative methodologies, as well as evaluation approaches and datasets currently employed in the domain.
- An in-depth discussion of robustness in diverse recommendation scenarios, its interplay with other trustworthy properties, and open issues and future research directions.

The remaining sections of this survey are organized as follows: In Section 2, we introduce the concept of robustness in recommender systems, defining what robustness means in this context and outlining a taxonomy for the field. Sections 3 and 4 provide a detailed discussion of state-of-the-art strategies for developing robust recommender systems, addressing both malicious attacks and natural noise, respectively. Section 5 explores the metrics and datasets commonly used to evaluate the robustness of recommender systems, and provides a comparative evaluation of representative robustness methods. In Section 6, we examine robustness considerations across various recommendation scenarios, while Section 7 investigates the interrelationship between robustness and other trustworthy properties of recommender systems, such as accuracy, interpretability, privacy, and fairness. Section 8 highlights open challenges and potential future directions in the study of robustness in recommender systems. Finally, Section 9 presents a succinct conclusion to this comprehensive survey.

2 Definition and Taxonomy

In this section, we provide a formal definition of robustness in the context of recommender systems and outline the taxonomy relevant to this domain. To establish a clear foundation, we first formalize recommendation tasks. Suppose we have a set of M users, denoted by $\mathcal{U}$, and a set of N items, denoted by $\mathcal{I}$. The rating set $\mathcal{R}$ contains feedback $r_{u,i}$ from user $u \in \mathcal{U}$ for item $i \in \mathcal{I}$. The collected user-item interactions are represented by $\mathcal{D}$, where $(u, i, r_{u,i}) \in \mathcal{D}$. The goal of recommender systems is to learn a parametric model $f : \mathcal{U} \times \mathcal{I} \rightarrow \mathcal{R}$ from $\mathcal{D}$ that minimizes the following objective function:

$$\mathcal{L}(\mathcal{D}) = \sum_{(u,i,r_{u,i}) \in \mathcal{D}} \psi(f(u,i), r_{u,i}), \quad (1)$$

where ψ is the error function used to measure the difference between the predicted and ground truth labels. For clarity, we provide a reference table summarizing the symbols used throughout this survey in Appendix Table 7. During model training, we typically split the collected data $\mathcal{D}$ into a training set $\mathcal{D}^+$ and a test set $\mathcal{D}^-$. After training on $\mathcal{D}^+$ by optimizing $\mathcal{L}(\mathcal{D}^+)$ in Equation (1), we obtain a recommender system model, denoted as $f_{\mathcal{D}^+}$.

2.1 Definition of Robustness in Recommender Systems

The term “robustness” initially described three essential characteristics of a parameterized model: *efficiency*, *stability*, and *non-breakdown* [58]. *Efficiency* refers to maintaining algorithmic efficiency in scenarios that align with model assumptions, stability demands minimal sensitivity to minor

deviations, and non-breakdown ensures performance does not collapse under substantial deviations. With the advancement of deep learning, robustness is generally defined as the ability of a model to maintain stable performance despite variations in relevant factors throughout the entire pipeline. Specifically, robustness can be categorized into three key phases of the machine learning pipeline:

- **Robustness in Model Training:** The ability to maintain performance despite perturbations or manipulations of training data or gradients, often referred to as poisoning attacks [60, 126]. In recommender systems, this involves altering training data (e.g., injecting fake users) to degrade performance or promote/nuke specific items [29, 122].
- **Robustness in Model Reasoning (Inference):** The capability of the model to consistently produce stable results for identical or closely related input contexts, also known as test-time perturbations. In recommender systems, this pertains to maintaining recommendation consistency despite slight variations in user or item features, interactions, or environmental conditions [50]. These variations may arise from attacks or noise introducing misleading recommendations.
- **Robustness in Model Evaluation:** The extent to which performance metrics remain stable under varying evaluation conditions, such as different metrics [133]. This dimension emphasizes the reliability and trustworthiness of evaluation results despite variations in data or metric choices.

Currently, certain recommendation paradigms, such as **collaborative filtering (CF)**, lack inference capabilities for new users and items until the model is explicitly updated. Consequently, they do not naturally support research on robustness in model reasoning (i.e., test-time perturbations). As a result, most robustness research in recommender systems primarily focuses on challenges during the training phase. Therefore, in this survey, we limit our discussion to **robustness concerning training-phase perturbations**. Additionally, in Section 5, we evaluate the effectiveness of various robustness methods across different types of metrics. Formally, we define robust recommender systems with respect to training-phase perturbations as follows:

Definition 2.1 ((ϵ, ϵ)-Robust Recommender Systems). Given a recommendation model f , a dataset $\mathcal{D}$ partitioned into a training set $\mathcal{D}^+$ and a test set $\mathcal{D}^-$, a perturbation budget ϵ , and an acceptable performance deviation ϵ , the recommender system is considered robust if:

$$|f_{\mathcal{D}^+}(\mathcal{D}^-) - f_{\mathcal{D}^+ + \Delta}(\mathcal{D}^-)| \leq \epsilon, \quad \forall \|\Delta\| \leq \epsilon, \quad (2)$$

where $f_{\mathcal{D}^+}$ denotes the model trained on clean training data, and $f_{\mathcal{D}^+ + \Delta}$ denotes the model trained on perturbed training data.

2.2 Taxonomy of Robustness in Recommender System

Recommender systems function as highly interactive platforms, making them vulnerable to various forms of abnormal data. These anomalies may stem from malicious activities, such as injecting fake users and tampering with item information, or from natural noise, which typically arises due to human errors or ambiguities in user behavior.

For malicious attacks, attackers often aim to promote/nuke specific items or degrade the performance of recommender systems. Generally, adversarial scenarios limit attackers' ability to manipulate a user's historical behavior. There are two primary types of attacks in recommendation scenarios, as shown in Figure 2(b): (1) **Item-side information modification:** Attackers alter item side information to artificially boost the popularity of specific items [32], as depicted at the top of Figure 2(b). (2) **Fake user injection (shilling attack):** Attackers introduce fake users to inflate or suppress the exposure of certain items or to degrade overall system performance [29, 122], as shown at the bottom of Figure 2(b).

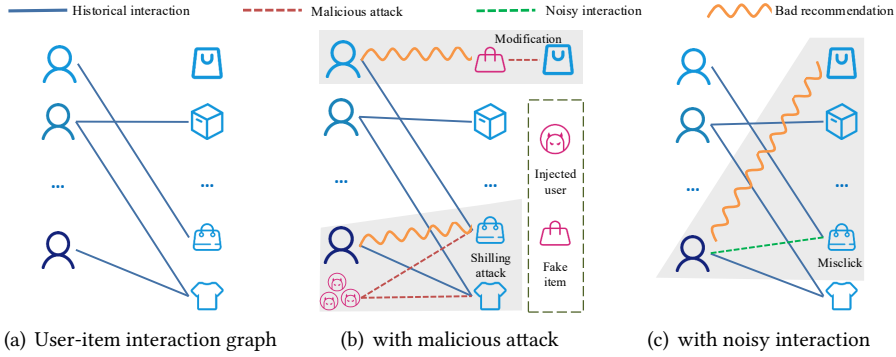


Fig. 2. User-item interaction graph with malicious attack and natural noise.

Natural noise primarily arises from user-generated factors such as human errors, uncertainty, and ambiguous user behavior [160]. For example, noise can manifest in user-item interactions, such as accidental clicks or gift purchases reflecting others' preferences, as illustrated in Figure 2(c). Additionally, noise may appear in user or item side information, such as incorrect personal details or mislabeled item tags. However, due to limited research addressing these types of noise in recommender systems [82], Figure 2(c) does not include this category.

To enhance the robustness of recommender systems, it is essential to distinguish between different types of abnormal data. Accordingly, we categorize recommender system robustness into two primary types:

- **Robustness against Malicious Attacks:** This category focuses on strategies to counteract injected users or falsified item data. Approaches are broadly divided into three types: (1) *Fraudster detection* [18, 148, 180]: Identifies fraudsters in training data and reduces their influence by either removing them or assigning lower weights; (2) *Adversarial training* [53, 121, 167]: Enhances model robustness by deliberately introducing small perturbations during training; (3) *Certifiable robust training* [83]: Trains models to function optimally under worst-case perturbations with formal robustness guarantees.
- **Robustness against Natural Noise:** This category addresses methods designed to mitigate the impact of noisy interactions. Techniques can be classified into three types: (1) *Regularization* [22, 169]: Introduces a regularization term (e.g., R1-norm regularization [169]) in the loss function to reduce the effect of noise; (2) *Purification* [125, 137]: Identifies and corrects noise in user-item interactions to enhance performance and robustness; (3) *Self-supervised learning (SSL)* [113, 140]: Mitigates noise through generative-based methods (e.g., **Denosing Autoencoder (DAE)** [77]) and contrastive-based approaches (e.g., contrastive learning [125]).

The taxonomy of publications on recommender system robustness is presented in Figure 3.

3 Robustness against Malicious Attack

In this section, we introduce various representative methods across three key categories—fraudster detection, adversarial training, and certifiable robust training—to enhance the robustness of recommender systems against malicious attacks.

3.1 Fraudster Detection

Detecting fraudsters is crucial for ensuring the robustness of recommender systems, particularly against malicious attacks. Such attacks often involve fraudsters providing misleading feedback to

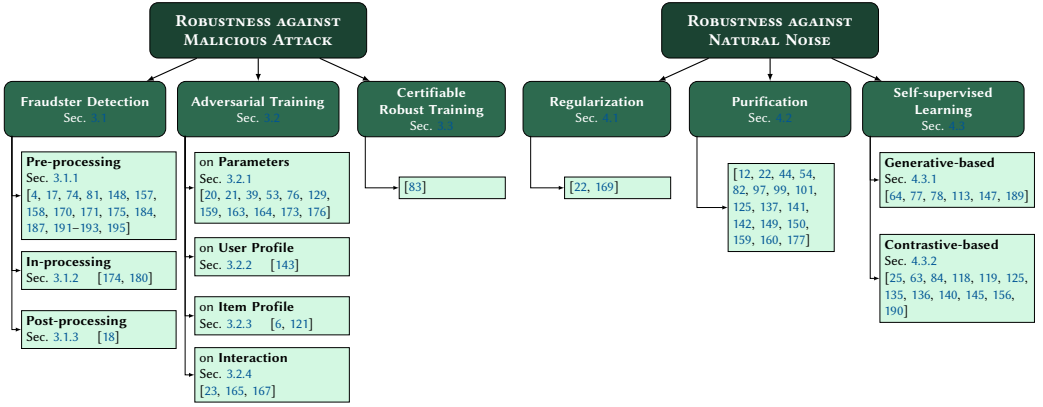


Fig. 3. A lookup graph for the reviewed methods on robustness in recommender systems.

manipulate the system [29, 122]. Therefore, fraudster detection aims to identify and eliminate these users to maintain reliable recommendations. Fraudster detection approaches can be categorized into three types based on the detection stage: pre-processing, in-processing, and post-processing detection (Figure 5(a)).

- **Pre-processing detection** [157, 193]: Identifies fraudsters in the original data before training, preventing their influence from the outset.
- **In-processing detection** [180]: Utilizes model feedback during training to detect and mitigate fraudster impact dynamically.
- **Post-processing detection** [18]: Identifies and corrects poor recommendations resulting from fraudsters after model training.

Each stage offers distinct detection strategies. While pre-processing is the most widely used approach, in-processing and post-processing techniques have gained attention in recent research.

3.1.1 Pre-Processing Detection. Pre-processing detection refers to identifying and mitigating fraudsters in recommender systems before model training, ensuring a reliable and accurate training process. It consists of two stages, of which the first stage is **Feature Extraction**, taking some statistics as the characteristics of each user; the second stage is **Detection**, through the features extracted in the first stage to detect fraudsters.

Feature Extraction. Most pre-processing detection methods rely on feature engineering to incorporate prior knowledge. Early studies [170, 171, 191] focused on developing user-centric features. For example, Zhou et al. [191, 192] introduced two indicators based on user attributes: Degree of Similarity with Top Neighbors (DegSim) and **Rating Deviation from Mean Agreement (RDMA)** [28]. DegSim quantifies a user’s similarity to their top k nearest neighbors using the average Pearson correlation:

$$DegSim_u = \frac{\sum_{v \in N_u @ k} W_{u,v}}{k}, \quad (3)$$

where $W_{u,v}$ is the Pearson correlation between user u and user v , and $N_u @ k$ represents the k nearest neighbors. RDMA measures a user’s rating deviation from the mean agreement of other users on target items, incorporating inverse rating frequency:

$$RDMA_u = \frac{\sum_{i \in \mathcal{I}_u} |r_{u,i} - \bar{r}_i| / |\mathcal{R}_i|}{|\mathcal{I}_u|}, \quad (4)$$

Table 1. Representative Generic Features in Fraudster Detection

Feature	Formula	Description	Publications
DegSim	$\frac{\sum_{v \in N_u @ k} W_{u,v}}{k}$	Degree Similarity Neighbors (DegSim) : Average Pearson Correlation of a profile's top k neighbors.	[17, 170, 187, 191, 192]
RDMA	$\sum_{i \in \mathcal{I}_u} \frac{ r_{u,i} - \bar{r}_i }{ \mathcal{I}_u }$	Rating Deviation Mean Agreement (RDMA) : Identifies attackers by examining average deviation and item ratings.	[17, 158, 170, 187, 191, 192]
WDA	$\sum_{i=0}^{N_u} \frac{ r_{u,i} - \bar{r}_i }{ \mathcal{R}_i }$	Weighted Degree Agreement (WDA) : Utilizes the RDMA numerator without considering profile rating count.	[157, 158, 170]
WDMA	$\sum_{i \in \mathcal{I}_u} \frac{ r_{u,i} - \bar{r}_i }{ \mathcal{I}_u }^2$	Weighted Deviation Mean Agreement (WDMA) : Based on RDMA, assigns higher weight to sparse item ratings.	[148, 157, 158, 170]
MeanVar	$\sum_{r_{u,i} \in \mathcal{R}_u - \mathcal{R}_{u,\max}} (r_{u,i} - \bar{r}_u)^2$	Mean-Variance (MeanVar) : Computes variance between filler items and overall average for attack detection.	[148, 157, 158]
LengthVar	$\frac{ \mathcal{R}_u - \mathcal{R}_{u,\max} }{\sum_{v \in U} (\mathcal{R}_v - \mathcal{R}_u)^2}$	Length-Variance (LengthVar) : Uses profile rating count to distinguish between genuine and attack profiles.	[17, 157, 158, 170]

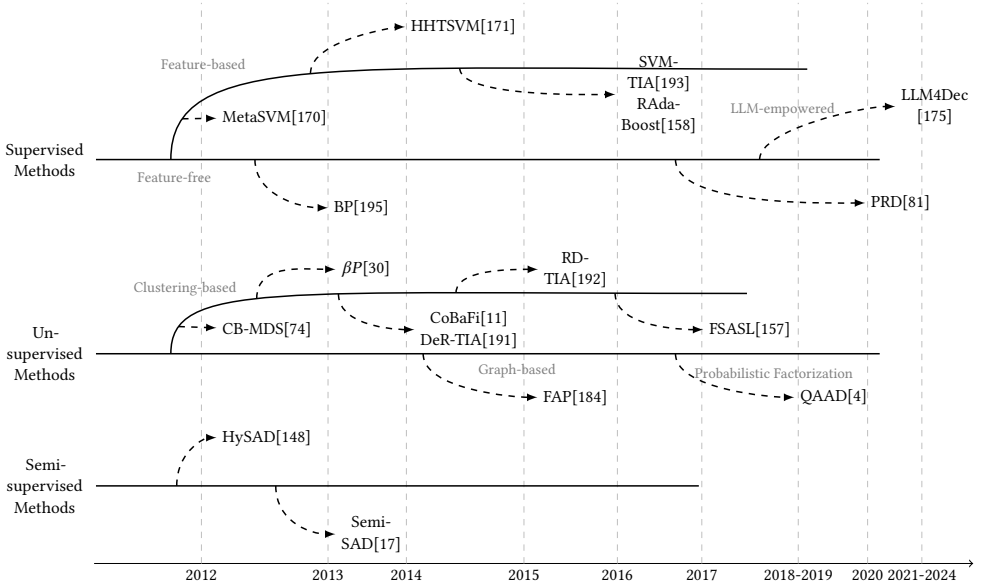


Fig. 4. Main development trajectory of pre-processing detection methods.

where $\mathcal{I}_u$ is the set of items rated by user u , $\bar{r}_i$ is the average rating for item i , and $|\mathcal{R}_i|$ is the number of interactions for item i . Typically, attackers exhibit higher RDMA values and lower DegSim values [191]. Additional features, such as **Weighted Deviation from Mean Agreement (WDMA)** [15, 157] and **Mean Variance (MeanVar)** [148, 158], further aid fraudster detection, particularly for filler items.⁴ Table 1 summarizes representative generic features used in pre-processing detection.

Detection. Detection methods for pre-processing can be broadly categorized into supervised, unsupervised, and semi-supervised learning approaches. To illustrate the evolution of these methods, Figure 4 presents their development trajectory.

⁴Filler items refer to randomly rated items in an injected user's profile in early non-optimized shilling attacks [51]. A broader definition includes any items in a user's profile that are neither the highest (push attack) nor the lowest (nuke attack) [15], which this survey adopts.

Supervised learning. Supervised fraudster detection relies on labeled datasets to train models for identifying fraudulent users. These datasets are typically constructed by simulating various attack methods and training models to recognize malicious activities. With feature extraction, classification models use feature vectors $\mathbf{x} = \{x_1, x_2, \dots, x_n\}$ as follows:

$$y = f_c(\mathbf{x}), \quad (5)$$

where f_c can be an SVM-based classifier [170, 171, 193] (MetaSVM, HHTSVM, SVM-TIA) or an ensemble method such as AdaBoost [158] (RAdaBoost). Additionally, some feature-free methods rely on interaction-based approaches. BP [195] models the probability distribution of fraudsters and target items based on interactions, using Belief Propagation for inference. Inspired by **Bayesian Personalized Ranking (BPR)** [102], PRD [81] detects fraudsters by analyzing pairwise ranking distributions, mitigating sample imbalance issues without relying on pre-selected features. Furthermore, LLM4Dec [175] leverages open-world knowledge of **large language models (LLMs)** to enhance pre-processing detection using only user interaction data. Such feature-free approaches improve adaptability by reducing dependence on predefined features, offering resilience against evolving malicious tactics.

Unsupervised learning. Unsupervised fraudster detection does not require labeled data; instead, it identifies anomalies or patterns in user behavior. Initial clustering-based methods aim to distinguish fraudulent users from genuine ones based on feature distributions. As highlighted by Beutel et al. [11] (CoBaFi), detectable spammers often form their distinct clusters.

Clustering-based approaches are prevalent. CB-MDS [74] applies hierarchical clustering, while DeR-TIA and RD-TIA [191, 192] use k-means. Some methods assume specific cluster properties. β P [30] employs the β distribution to detect attackers with pre-selected features, leveraging statistical properties to identify suspicious patterns. FSASL [157] capitalizes on the tendency of malicious users to mimic genuine users, resulting in higher-density clusters. Similar density-based clustering methods, such as DBSCAN and LOF [13], effectively identify densely connected groups of fraudulent users.

Beyond clustering, other strategies exploit the inherent structure of user-item interactions. FAP [184] detects anomalies by analyzing the graph structure of recommendations. QAAD [4] introduces a probabilistic factorization model to estimate rating likelihoods, assuming that genuine users exhibit higher likelihoods than fraudsters, whose fabricated profiles often show inconsistencies.

Semi-supervised learning. Semi-supervised learning leverages a small labeled dataset to train an initial classifier, then iteratively improves it by incorporating unlabeled data [148]. This approach maximizes the utility of limited labeled data while utilizing large volumes of unlabeled data to enhance detection performance. Notable examples include HySAD [148] and Semi-SAD [17], which initially train a basic classifier on limited labels and refine it using the **Expectation-Maximization (EM)** algorithm. By learning from unlabeled samples, the classifier progressively improves its accuracy. Other semi-supervised methods, such as self-training, co-training, and multi-view learning, could be explored for fraudster detection in the future.

Conclusion. Each pre-processing detection approach offers unique advantages but also presents challenges. Supervised methods struggle with data imbalance, as genuine users far outnumber fraudsters. Unsupervised techniques, lacking explicit guidance, may fail to distinguish fraudsters who closely resemble genuine users. Despite these challenges, solutions from other domains can be adapted. For instance, graph-based anomaly detection algorithms [56, 57, 90] leverage the network structure formed by users and items to identify fraudsters. Advanced machine learning techniques, including deep learning [72] and **reinforcement learning (RL)** [120], could further enhance detection by capturing complex behavioral patterns.

3.1.2 In-Processing Detection. In-processing detection refers to identifying and mitigating fraudulent activities in recommender systems during model training. Unlike pre-processing detection, this approach leverages both the initial data and real-time training information from the training process, capturing complex and latent relationships among users and items. In-processing detection [174, 180] can be formulated as two tasks:

Fraudster Detection: Leveraging information from the training process enables more accurate fraudster detection. GraphRfi [180] incorporates principles from cognitive psychology, assuming that genuine users exhibit coherent and predictable behavior. If a user's actual behavior significantly deviates from their predicted behavior, they are likely a fraudster. LoRec [174] is the first to leverage LLMs for fraudster detection in recommender systems, enabling generalization beyond specific attack patterns and enhancing robustness against unknown threats. It further integrates item embeddings from the recommender system with user representations derived from sequential behavior modeling to capture nuanced fraud patterns. The fraudster detection task is formalized as

$$\mathcal{L}_{\text{detection}} = \mathbb{E}_u [-\log \mathbb{P}[y = y_u | a_u]], \quad (6)$$

where $\mathbb{P}[y = y_u | a_u]$ represents the probability of user u being classified as a fraudster ($y_u = 0$) or a genuine user ($y_u = 1$), and a_u denotes auxiliary information obtained during the training process.

Robust Recommendation: By incorporating fraudster probability values from the detection phase, GraphRfi [180] and LoRec [174] dynamically adjust the training weight of each user to mitigate attack impacts. This is achieved through the following objective function:

$$\mathcal{L}_{\text{rec}} = \mathbb{E}_u \left[\sum_{i \in \mathcal{I}} \mathbb{P}[y = 1 | a_u] \cdot \mathcal{L}(u, i) \right], \quad (7)$$

where $\mathcal{L}(u, i)$ represents the loss associated with user u and item i .

3.1.3 Post-Processing Detection. Post-processing detection occurs after the recommender system has been trained. Its primary objective is to filter out low-quality recommendations influenced by fraudsters, ensuring that the system provides accurate and reliable suggestions to genuine users.

A representative post-processing detection method was proposed by Cao et al. [18]. In the RL-based recommendation setting, they introduce a two-part detection model. The first component is a **Gated Recurrent Unit (GRU)** encoder, which encodes the action sequences—i.e., the recommendation lists generated by the RL agent—into a low-dimensional feature vector. The second component is an attention-based decoder with a classifier that differentiates between high-quality and low-quality recommendations. This method effectively identifies poor-quality recommendations resulting from fraudulent activities. The RL-based framework enables the model to continuously refine its decision-making process by learning from previous filtering results, thereby enhancing the overall robustness of the recommender system's output.

3.1.4 Discussion of Fraudster Detection. In the context of fraudster detection in recommender systems, there are three principal strategies: pre-processing, in-processing, and post-processing detection. Pre-processing detection operates before model training, offering the advantage of computational efficiency during training [4, 74]. In-processing detection integrates model insights throughout training to achieve improved accuracy [180]. Meanwhile, post-processing detection aims to filter out poor recommendations for subsequent items [18], though its performance may be compromised when filtering entire recommendation lists.

The tradeoff between precision and recall is a crucial challenge in fraudster detection. Lowering the classification threshold may increase fraudster detection rates but risks misidentifying genuine users, negatively affecting user experience and financial outcomes. Thus, future detection methods should prioritize maintaining high precision while improving recall. Exploring advanced deep

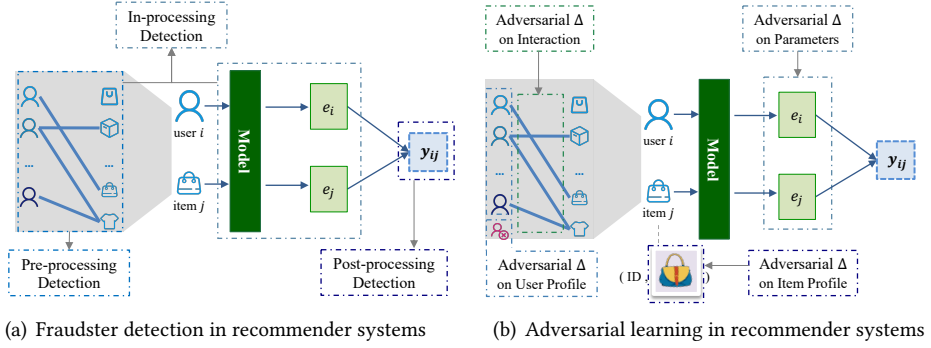


Fig. 5. Fraudster detection and adversarial learning.

learning techniques, such as graph neural networks, appears promising for identifying complex patterns and relationships between users and items, which could further enhance detection.

3.2 Adversarial Training

Adversarial training has emerged as a promising approach to enhance the robustness of deep models against various malicious attacks [48, 166]. This technique introduces small, carefully crafted perturbations to input data during training, improving the model's resilience to adversarial samples [29, 122]. By learning from these adversarial samples, recommender systems can better generalize to unseen or manipulated data, thereby enhancing overall robustness and reliability.

Adversarial training in recommender systems can be categorized based on the nature of the perturbations introduced. Specifically, it can target model parameters, user profiles, item profiles, or user-item interactions. These categories are graphically illustrated in Figure 5(b). To provide a comprehensive view of the evolution of each category, we depict their development trajectories in Figure 6. In the following subsections, we delve into each category, providing a comprehensive overview of the representative works.

3.2.1 Adversarial Perturbation on Parameters. Most input features in recommender systems, such as user and item IDs or categorical attributes, are discrete. As a result, even minor perturbations can significantly alter their semantics [53]. This discreteness poses challenges for adversarial training, which typically enhances robustness by perturbing input data, as commonly done in computer vision [48]. To address this limitation, He et al. [53] proposed **Adversarial Personalized Ranking (APR)**, which introduces perturbations at the parameter level to simulate the effects of malicious attacks. The objective function is formulated as

$$\Theta^* = \arg \min_{\Theta} \max_{\Delta, \|\Delta\| \leq \epsilon} \mathcal{L}(\mathcal{D}|\Theta) + \lambda \mathcal{L}(\mathcal{D}|\Theta + \Delta), \quad (8)$$

where $\mathcal{L}$ represents the personalized ranking loss function, Θ denotes the model parameters, Δ represents perturbations on model parameters, and $\epsilon \geq 0$ controls the perturbation magnitude.

Subsequent research [20, 76, 129, 159] has further developed adversarial training in recommender systems, particularly focusing on parameter perturbations. ATF [20] integrates the Pairwise Interaction Tensor Factorization [103] into APR, enhancing context-aware recommendations [9]. Similarly, APC [129] and SACRA [76] build upon APR in their models to improve robustness. RocSE [159] further considers the structure of the interaction graph, while SharpCF [21] reduces computational complexity by transforming adversarial training into Sharpness-Aware Training.

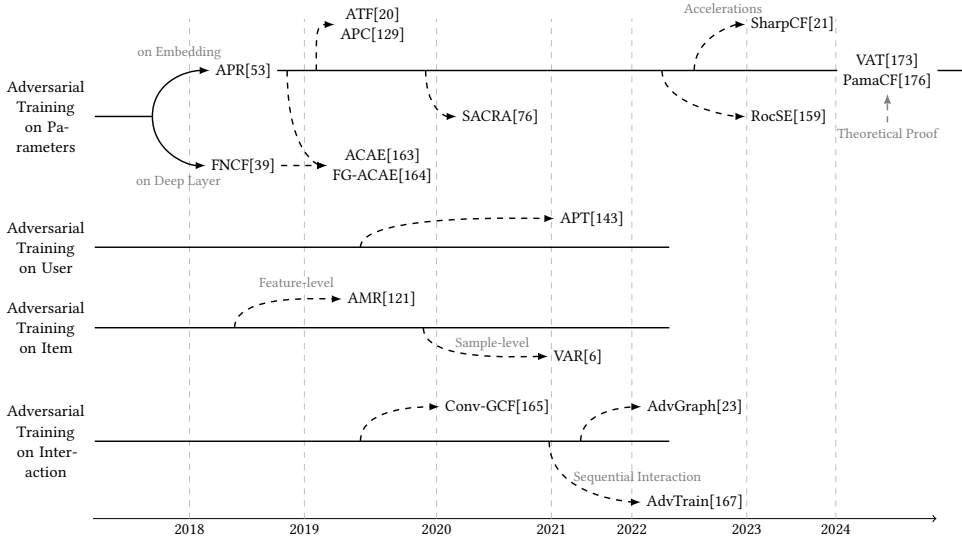


Fig. 6. Main development trajectory of adversarial training methods.

Notably, VAT [173] and PamaCF [176] introduce differentiated perturbations to enhance adversarial training. VAT explores user vulnerability and employs vulnerability-aware adversarial training to simultaneously mitigate performance degradation and strengthen defense capabilities. PamaCF provides a theoretical analysis explaining how adversarial training improves both robustness and overall performance in recommender systems. Leveraging these insights, PamaCF further optimizes adversarial CF performance.

Taking a different approach, FNCF [39] investigates the effects of parameter perturbations across various layers of neural network-based recommender systems, shedding light on how deeper layers respond to perturbations. Bridging different techniques, ACAE [169] and FG-ACAE [164] combine principles from APR [53] and FNCF [39], addressing perturbations in both user/item embeddings and deeper model layers.

3.2.2 Adversarial Perturbation on User Profile. Adversarial training methods focusing on user profiles take a defensive stance against attacks, aiming to protect recommender systems from various adversarial threats. A straightforward approach involves injecting specific users during training to enhance the model’s robustness against potential malicious users. The objective function is formulated as

$$\Theta^* = \arg \min_{\Theta} \max_{\Delta, \|\Delta\| \leq \epsilon} \mathcal{L}(\mathcal{D}|\Theta) + \lambda \mathcal{L}(\mathcal{D} + \Delta|\Theta), \quad (9)$$

where $\mathcal{D} + \Delta := (\mathcal{U} \cup \mathcal{U}_{inj}, \mathcal{I}, \mathcal{R} \cup \mathcal{R}_{inj})$.

Here, $\mathcal{U}_{inj}$ denotes the set of injected users, while $\mathcal{R}_{inj}$ is the interactions associated with them.

Although injecting adversarial users can strengthen the model’s robustness, it does not necessarily mitigate the impact of pre-existing malicious users within the training data. This raises a critical question: How can we counteract the effects of already poisoned training data? To address this, Wu et al. [143] proposed **Adversarial Poisoning Training (APT)**, which follows a “fight fire with fire” strategy. Instead of simply removing malicious users, APT introduces **empirical risk minimization (ERM)** users, whose profiles are designed to counterbalance the effects of adversarial

users. The corresponding objective function is

$$\Theta^* = \arg \min_{\Theta} \min_{\mathcal{D}_{\text{ERM}}, |\mathcal{D}_{\text{ERM}}|=n} \mathcal{L}(\mathcal{D} \cup \mathcal{D}_{\text{ERM}}|\Theta), \quad (10)$$

where $\mathcal{D}_{\text{ERM}}$ represents the set of user profiles that minimize empirical risk, and n specifies the number of ERM users introduced. By adding ERM users to the training dataset, APT improves the recommender system's robustness, effectively mitigating the effects of malicious users.

3.2.3 Adversarial Perturbation on Item Profile. Content-based recommendation leverages rich side information to improve recommendation accuracy. However, this approach is vulnerable to external threats, where malicious entities may manipulate item side information to unfairly promote certain products or distort recommendations. To counteract such adversarial actions, researchers have introduced adversarial perturbations in handling item side information, either in the original data domain [6] or in the latent feature space [121]. The objective function is

$$\begin{aligned} \Theta^* &= \arg \min_{\Theta} \max_{\Delta, \|\Delta\| \leq \epsilon} \mathcal{L}(\mathcal{D}|\Theta) + \lambda \mathcal{L}(\mathcal{D} + \Delta|\Theta), \\ \text{where } \mathcal{D} + \Delta &:= (\mathcal{U}, \mathcal{I} + \Delta, \mathcal{R}), \end{aligned} \quad (11)$$

where $\mathcal{I} + \Delta$ represents perturbations applied either in the original feature space or the latent space of item side information. The choice between these approaches depends on the primary objective—whether to enhance robust feature extraction⁵ or to strengthen recommender systems against adversarial vulnerabilities.

3.2.4 Adversarial Perturbation on Interaction. In recommender systems, adversarial training on interaction graphs has gained increasing attention. These methods introduce adversarial perturbations within interaction data to enhance the system's robustness against potential threats targeting user-item interactions. Recent studies, such as Conv-GCF [165] and AdvGraph [23], embed adversarial perturbations directly into the interaction matrix. This approach can be formulated as the following optimization problem:

$$\begin{aligned} \Theta^* &= \arg \min_{\Theta} \max_{\Delta, \|\Delta\| \leq \epsilon} \mathcal{L}(\mathcal{D}|\Theta) + \lambda \mathcal{L}(\mathcal{D} + \Delta|\Theta), \\ \text{where } \mathcal{D} + \Delta &:= (\mathcal{U}, \mathcal{I}, \mathcal{R} + \Delta), \end{aligned} \quad (12)$$

where $\mathcal{R} + \Delta$ represents the perturbed interactions.

Beyond static interaction data, AdvTrain [167] explores adversarial training in sequential recommendation settings. Their approach perturbs interaction sequences, which is particularly relevant for sequential recommendation systems, where user temporal behavior significantly influences recommendation outcomes.

3.2.5 Discussion of Adversarial Training. Adversarial training has introduced various strategies to enhance the robustness of recommender systems. Among these, *Adversarial Perturbation on Parameters* has gained significant attention as a prevalent approach. By introducing perturbations at the parameter level, this method equips models to counteract disruptive manipulations from malicious users [20, 53]. *Adversarial Perturbation on User Profile* adopts a defensive strategy similar to that of attackers. By injecting users into the system, it strengthens the model's resistance against adversarial manipulation [143]. In contrast, *Adversarial Perturbation on Item Profile* aligns more closely with content-based recommendation frameworks, focusing on perturbations related to item-specific attributes [6, 121]. Finally, *Adversarial Perturbation on Interaction* introduces discrete

⁵Feature Extraction: the process of obtaining representations of item side information [6].

perturbations in user-item interactions to enhance model resilience against interaction-level attacks [23, 167]. However, given the limited adversarial capabilities in practice, interaction-level attacks are relatively rare in real-world recommendation scenarios.

Future research could explore synergies between different adversarial perturbation strategies, such as those targeting user profiles, item profiles, and system parameters, to develop comprehensive defense mechanisms against diverse threats. Additionally, designing novel adversarial training methodologies that adapt to the dynamic nature of modern recommender systems presents an exciting avenue for further investigation.

3.3 Certifiable Robust Training

Certifiable robust training plays a crucial role in ensuring the integrity and reliability of recommender systems against malicious attacks. This approach focuses on designing algorithms that provide formal guarantees of robustness against adversarial perturbations, ensuring that recommendations remain accurate even in the presence of malicious users.

Liu et al. [83] exemplify this approach by defining the robust boundary of the **Factorization Machine (FM)** model. Given a trained FM, denoted as f , and an input instance $\mathbf{x}$ with d -dimensional features in the binary space $\{0, 1\}^{1 \times d}$, the FM-based recommendation task with second-order weights is expressed as

$$f(\mathbf{x}) = w_0 + \sum_{i=1}^d w_i x_i + \sum_{i=1}^d \sum_{j=i}^d \langle v_i, v_j \rangle x_i x_j, \quad (13)$$

where w_0 is the global bias, w_i represents the weight of the i th feature, $v_i \in \mathbb{R}^{1 \times k}$ is the embedding of the i th feature, and the inner product $\langle v_i, v_j \rangle$ captures interactions between features i and j . The variable x_i corresponds to the i th dimension of instance $\mathbf{x}$.

With a perturbation budget p , which limits the maximum number of feature flips in $\mathbf{x}$, Liu et al. [83] estimate the upper bound of prediction alteration, denoted as $b(\mathbf{x})$. If $f(\mathbf{x}) \leq 0$, then $b(\mathbf{x}) = \max_{\mathbf{x}'} f(\mathbf{x}') - f(\mathbf{x})$; otherwise, $b(\mathbf{x}) = \min_{\mathbf{x}'} f(\mathbf{x}') - f(\mathbf{x})$. Here, $\mathbf{x}'$ represents a perturbed version of $\mathbf{x}$, constrained by the condition $|\mathbf{x} \oplus \mathbf{x}'| \leq p$, where $\oplus$ denotes the XOR operation. This measures the model's output shift under the maximum allowable perturbation. Evaluating whether the prediction changes significantly under this bound provides insight into the model's robustness, as further discussed in Section 5.1. Building on this foundation, Liu et al. [83] propose a robust training algorithm formulated as

$$\Theta^* = \arg \min_{\Theta} \sum_{\mathcal{D}} \log [1 + \exp((-y)(f(\mathbf{x}) + b(\mathbf{x})))] , \quad (14)$$

where $f(\mathbf{x}) + b(\mathbf{x})$ represents the prediction bound under the maximum perturbation shift.

Certifiable robust training provides a valuable framework for assessing and enhancing the robustness of recommender systems, allowing developers to better safeguard their models against adversarial attacks while maintaining accurate and reliable recommendations.

4 Robustness against Natural Noise

Recommender systems also encounter challenges from natural noise, which arises due to inconsistencies, inaccuracies, or missing information in input data. Such noise can stem from various sources, including human error, uncertainty, and vagueness [160]. Ensuring robustness against natural noise is essential for maintaining accurate and reliable recommendations. To mitigate this issue, researchers have explored various techniques, including regularization, purification, and SSL. This section provides a detailed discussion of these methods, highlighting their contributions to enhancing the robustness of recommender systems against natural noise.

4.1 Regularization

Regularization is a widely used technique in machine learning to prevent overfitting by constraining model complexity. By limiting the model's capacity to learn intricate and fluctuating noise patterns, regularization enhances generalization and improves robustness against natural noise in input data. Mathematically, the regularization objective is expressed as

$$\Theta^* = \arg \max_{\Theta} \mathcal{L}(\mathcal{D}|\Theta) + \lambda \|\Theta\|_k, \quad (15)$$

where $\mathcal{L}(\mathcal{D}|\Theta)$ denotes the model's loss function based on dataset $\mathcal{D}$ and parameters Θ . The term $\|\Theta\|_k$ represents the regularization penalty, determined by the L_k norm of the model parameters, and λ is a hyperparameter that balances model fit and complexity.

Beyond simple parameter constraints, Zhang et al. [169] introduced R_1 -norm⁶ regularization on predictions to reduce sensitivity to noise. Their loss function is formulated as

$$\Theta^* = \arg \max_{\Theta} \sum_{i \in \mathcal{I}} \sqrt{\sum_{u \in \mathcal{U}} (r_{u,i} - \hat{r}_{u,i})^2} + \|\Theta\|_F^2, \quad (16)$$

where $r_{u,i}$ is the ground truth rating, and $\hat{r}_{u,i}$ is the predicted rating computed by Θ . The R_1 -norm provides greater robustness against outliers compared to the Euclidean distance (i.e., L_2 -norm). By integrating R_1 -norm regularization, the model prioritizes essential features over noise, leading to more refined recommendations. Recent research continues to advance regularization strategies for enhancing robustness in noisy recommender systems. For instance, Chen et al. [22] combine Jacobian regularization [68] with transformer blocks in sequential recommendation models. This approach significantly reduces the model's susceptibility to noisy sequences, resulting in more stable and reliable recommendations.

While regularization is a general and effective technique for preventing overfitting and enhancing generalization, it often fails to directly address the root causes of specific types of noise in recommender systems. As a result, it is typically used in conjunction with other noise-mitigation strategies to achieve optimal robustness.

4.2 Purification

Purification is an effective technique targeted for identifying and rectifying noise in user-item interactions, thereby enhancing the performance and robustness of recommender systems. The focus of this approach is to detect and eliminate noise from the input data during the training process to account for the presence of noise. The primary objective of purification can be mathematically formulated as follows:

$$\Theta^* = \arg \min_{\Theta} \mathcal{L}(\mathcal{D}|\Theta, W), \quad (17)$$

where W is the weight matrix for all interactions. More specifically:

$$\mathcal{L}(\mathcal{D}|\Theta, W) = \mathbb{E}_{(u,i,r)} [w_{u,i} \psi(f(u,i), r_{u,i})], \quad (18)$$

where $w_{u,i}$ is the weight of interaction $r_{u,i}$ in W . To provide a comprehensive understanding of the development of the methods based on purification, we illustrate their trajectories in Figure 7.

Based on User Profiles. Historically, various works have proposed strategies to identify noisy interactions based solely on user profiles. IMDB [97] integrates interactions and item-related information to formulate a user preference model, marking deviations from anticipated user preferences as anomalies. Moving forward, PV [127] proposes an innovative preference model that obviates the need for supplementary information. In a different approach, NN-Fuzzy [160]

⁶The R_1 -norm of a matrix $X \in \mathbb{R}^{d \times l}$ is defined as $\|X\|_{R_1} = \sum_{j=0}^l \sqrt{\sum_{i=0}^d x_{i,j}^2}$

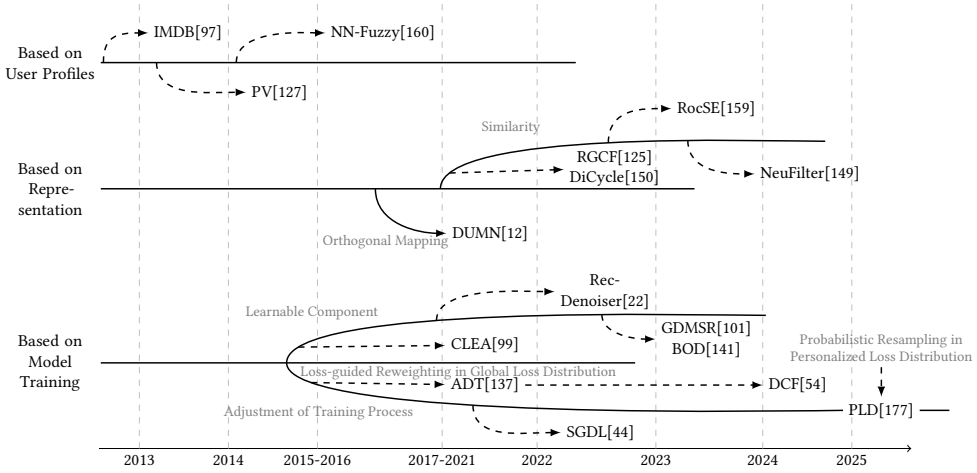


Fig. 7. Main development trajectory of purification methods.

introduces a fuzzy inference system to detect noisy ratings, leveraging uncertainty modeling to refine preference predictions.

Based on Representations. More recently, methods have been proposed that leverage learned representations to re-weight noisy interactions. DUMN [12] leverages the representations of explicit feedback (devoid of noise) to refine the representations of implicit feedback (potentially tainted with noise) through orthogonal mapping. Both RGCF [125] and RocSE [159] measure the congruence between user-item pairs to identify noise, considering interactions with minimal similarity as potential noise. DiCycle [150] further considers the dynamics of interaction. By employing a continuous translation-invariant kernel [152], it translates timestamps with item representation into embeddings. The resulting inner product of the embeddings of timestamps t_i and t_j reflects the consistency of user interactions across these timestamps, positing that inconsistent interactions could be tinged with noise. Additionally, NeuFilter [149] introduces Kalman Gain to estimate user representations, thereby mitigating the unreliability caused by noisy interactions. These adaptive estimates enhance robustness against varying noise patterns.

Based on Model Training. Another paradigm in noise purification focuses on model training. Several methodologies integrate learnable components into the recommendation model to spotlight noise [22, 99]. CLEA [99] and GDMSR [101] use a **multi-layer perceptron (MLP)** to detect the noisy items in historical interactions during training. GDMSR [101] further integrates Dunbar’s number theory to adaptively adjust noise levels for each user based on the upper limit of social relationships, thereby incorporating social constraints into the denoising process. Meanwhile, Rec-Denoiser [22] integrates a differentiable mask in its attention mechanism, detecting potential noisy interactions. Additionally, BOD [141] directly learns interaction-specific weights to refine the denoising process, offering a more granular noise-adjustment mechanism.

Concurrently, other researches [44, 137] detect noise by observing the data pattern in the training phase. Both ADT [137] and SGDL [44] posit that clean and noisy interactions reveal distinct patterns throughout training. In particular, ADT utilizes a truncated loss to eliminate noisy interactions, while SGDL employs meta-learning to fine-tune the denoising process. Moreover, DCF [54] extends ADT by preserving hard examples with relatively high losses. Furthermore, PLD [177] innovatively leverages users’ personalized loss distributions to adjust the training sampling strategy, enhancing

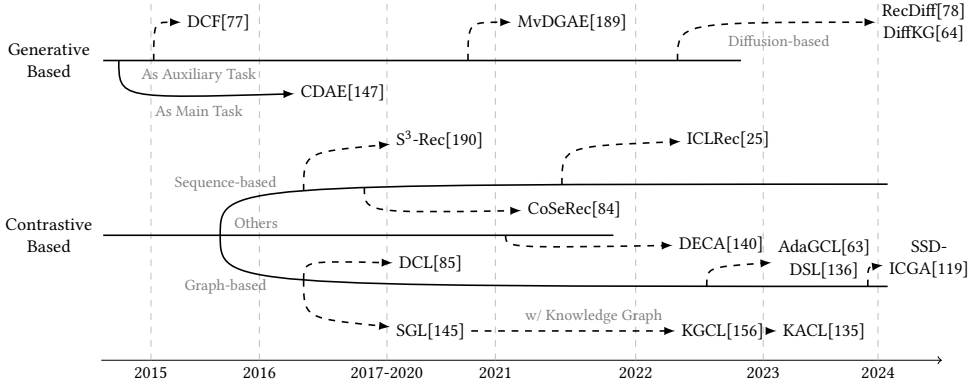


Fig. 8. Main development trajectory of self-supervised methods.

both efficiency and performance in denoising. By tailoring the denoising process to individual users, PLD adapts to diverse user behaviors, reducing over-filtering risks.

Conclusion. The purification technique is a powerful method for enhancing the robustness of recommender systems by identifying and correcting noise in user-item interactions. However, it is a challenging task that requires careful design and selection of appropriate models and strategies, considering the specific characteristics of the recommendation scenarios and noise types.

4.3 Self-Supervised Learning

SSL has emerged as a powerful machine learning paradigm, extensively applied in scenarios where explicit labels are scarce or unavailable [80, 108]. This approach exploits the inherent structure of the data [80] or generates data variants [61] to enhance generalization capabilities. In recommender systems, self-supervised strategies can be broadly classified into two categories: generative-based methods [77, 147, 189] and contrastive-based methods [145, 151, 190]. Generative-based methods corrupt input data and use the restoration process as a learning signal, whereas contrastive-based methods generate multiple data views through augmentation. The model is trained to extract essential features across these views while minimizing noise by maximizing mutual information between different views. These methods play a crucial role in mitigating the impact of noise in user-item interactions and auxiliary information [55, 161].

To provide a comprehensive overview of the development of SSL techniques in robust recommender systems, we illustrate their progression in Figure 8. Given the extensive research on SSL in recommendation, we focus on methods explicitly designed for denoising. For a broader view of self-supervised methods, we refer readers to the survey [161].

4.3.1 Generative-Based Self-Supervised Learning. DAEs [132] serve as a cornerstone of generative-based SSL in recommender systems. These methods operate on **masked** user or item side information [77], user-item interaction matrices [147], or other attributes [189], aiming to reconstruct the original information from corrupted inputs.

Consider a matrix $X \in \{0, 1\}^{m \times n}$, which could represent attributes [77], interactions [147], or other features [189]. By applying a mask $M \in \{0, 1\}^{m \times n}$, the corrupted matrix is generated as $\tilde{X} = X \odot M$, where $\odot$ denotes element-wise multiplication. Certain techniques leverage the restoration task as a supplementary objective. For instance, in DCF [77], a DAE, symbolized as g , is integrated into recommender systems to revive the original attributes matrix X from its corrupted

version, $\tilde{X}$. Expanding upon this, MvDGAE [189] delves into both user and item interconnections. Specifically, MvDGAE is dedicated to reconstructing both the intricate relationships among users and the coexistence patterns between items. These methods can be mathematically articulated as

$$\Theta^*, g^* = \arg \min_{\Theta, g} \mathcal{L}(\mathcal{D}|\Theta) + \underbrace{\lambda \|g(\tilde{X}) - X\|_F^2}_{\text{Reconstruction}} \quad (19)$$

where Θ is composed of the latent features in g .

Additionally, with the rapid advancement of diffusion models, recent studies such as RecDiff [78] and DiffKG [64] have begun leveraging diffusion-based denoising capabilities to enhance recommender system robustness. These methods iteratively refine representations through a progressive noise removal process, leading to improved denoising capability.

Conversely, an alternative avenue focuses on deploying DAEs to generate recommendations, aligning with the principles of auto-encoder-based recommender systems [110]. In CDAE [147], the term $\mathcal{L}(\mathcal{D}|\Theta)$ can be defined as

$$\mathcal{L}(\mathcal{D}|\Theta) = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \underbrace{\|g_\Theta(\tilde{X}_u) - X_u\|_F^2}_{\text{Reconstruction}} \quad (20)$$

where X_u denotes the interactions of user u , and g_Θ denotes the DAE-based recommendation model.

4.3.2 Contrastive-Based Self-Supervised Learning. Contrastive learning maximizes mutual information between positive pairs—different augmented views of the same sample. Applied to recommender systems, this approach is formalized as

$$\Theta^* = \arg \min_{\Theta} \mathcal{L}(\mathcal{D}|\Theta) - \underbrace{\lambda \mathbb{E}_{u \in \mathcal{U}} [I(v_u, v'_u)|\Theta] \text{ or } \lambda \mathbb{E}_{i \in \mathcal{I}} [I(v_i, v'_i)|\Theta]}_{\text{Contrastive}} \quad (21)$$

where $I(\cdot, \cdot)$ denotes mutual information. Common loss functions include InfoNCE [95] and Cross-Entropy. Positive pairs (v, v') originate from different augmentation strategies, primarily categorized as sequence-based or graph-based.

Sequence-based. In the domain of sequence-based contrastive learning, pioneering methods like S³-Rec [190] emphasize aligning items with their intrinsic attributes. This is achieved by masking either attributes, items, or specific segments of sequences, thus engendering diverse views tailored for contrastive learning. Furthering this approach, CoSeRec [84] innovatively substitutes and inserts items into historical item sequences based on item co-appearances in interaction sequences. Additionally, ICLRec [25] delves into the relationship between historical interactions and a user's shopping intent, guaranteeing a congruous connection between sequence views and their intent.

Graph-based. Contrastive learning techniques based on interaction graphs, such as SGL [145], employ diverse graph augmentations, including node dropping, edge masking, and sampling distinct subgraphs via random walks. Similarly, DCL [85] utilizes stochastic edge masking to perturb specific network structures, leading to dual augmented neighborhood subgraphs. However, random augmentations in SGL may compromise the preservation of useful interactions for contrastive learning, as pointed out by KGCL [156]. To address this, KGCL integrates supplementary knowledge graph to improve the masking approach. Furthermore, KACL [135] refines this approach by introducing an adaptive view generator that dynamically selects important edges, effectively filtering out irrelevant information and mitigating knowledge overload in recommender systems.

Beyond knowledge-based enhancements, various methods leverage different assumptions to construct more robust augmented views. For instance, AdaGCL [63] incorporates a graph variational autoencoder along with a constrained denoising matrix to generate diverse views, thereby improving robustness. DSL [136] constructs denoised views by aligning user interaction similarity with social

Table 2. Comparison of Different Evaluation Methods

Evaluation Method	Formula	Efficient	Certifiable	Model-agnostic	Publication
Offset on Metrics	$\Delta M = \frac{ M' - M }{M}$	✓		✓	[6, 22, 25, 37, 39, 53, 77, 84, 115, 118, 121, 125, 131, 140, 143, 159, 163–165, 167, 173–177, 180]
Offset on Output	$\Delta O = E_u [\text{sim}(\hat{L}_u @ k, \hat{L}'_u @ k)]$	✓		✓	[93, 116, 143]
Certifiable Robustness	$\delta = f(\mathbf{x}') - f(\mathbf{x})$		✓		[83]

relationship similarity, reducing noise from both perspectives. Additionally, SSD-ICGA [119] applies the Independent Cascade Graph model to simulate influence diffusion, generating enhanced graph views by filtering out inactive edges, which improves noise reduction.

Additionally, the seminal work by Wang et al. [140] highlights the consistency in clean sample predictions across models, contrasting it with the variability observed in noisy samples. Building on this observation, DeCA [140] employs multiple models as a means of data augmentation, resulting in enhanced contrastive learning effectiveness.

In conclusion, the incorporation of SSL into recommender systems stands as a potent remedy for inaccuracies induced by noise. Beyond the aforementioned methods, various self-supervised strategies originally developed for graph data—whether static [88, 194] or dynamic [106, 172]—can be effectively adapted to recommendation scenarios.

Apart from the aforementioned approaches, several self-supervised strategies specifically designed for graphs, be they static [88, 194] or dynamic [106, 172], can be aptly adapted to recommendation scenarios. These techniques bolster the model’s ability to extract resilient and flexible representations, which are paramount for delivering superior recommendations in real-world settings. These techniques enhance the model’s capacity to learn robust and flexible representations, which are essential for delivering high-quality recommendations in real-world applications.

5 Evaluation

In this section, we discuss the evaluation of recommender systems, focusing on two key aspects: the evaluation methods for robustness in recommender systems and the commonly used datasets for assessment. Furthermore, we perform experimental comparisons and analyses of representative robustness methods, illustrating their effectiveness across various recommendation scenarios.

5.1 Evaluation Methods of Robustness in Recommender Systems

The average prediction shift [16] between the clean training set and the perturbed training set is commonly used to evaluate the robustness of a given recommender system. In this section, we introduce three types of commonly used evaluation methods for assessing recommender systems’ robustness, as listed in Table 2. Note that we only include publications that directly employ one of these three methods in the table. For instance, some approaches analyze the variation in model performance as noise levels or attack intensity change. While this approach is a variant of *Offset on Metrics*, it does not directly present quantitative evaluation indicators but rather trends. Therefore, we do not list such publications in Table 2.

5.1.1 Offset on Metrics. To efficiently evaluate the robustness of recommender systems, some [39, 121, 173, 176, 177] measure the offset on performance metrics as an indirect representation of prediction shift, formulated as follows:

$$\Delta M = \frac{|M' - M|}{M}, \quad (22)$$

where M represents the performance of the recommender system trained on clean data, and M' represents the performance of the system trained on perturbed data. For example, several studies [37, 115, 164, 165] utilize incremental $\text{HR}@k$ ⁷ to compute the shift as $\Delta_{\text{HR}@k} = |\text{HR}@k' - \text{HR}@k|$. Similarly, other works [25, 84, 159] assess robustness using the offset on $\text{NDCG}@k$ ⁸, computed as $\Delta_{\text{NDCG}@k} = |\text{NDCG}@k' - \text{NDCG}@k|$. Furthermore, Wu et al. [143] propose a refined method, **Robustness Improvement (RI)**, which better evaluates defense mechanisms:

$$\text{RI} = 1 - \frac{\text{HR}@k' - \text{HR}@k}{\text{HR}@k^* - \text{HR}@k}, \quad (23)$$

where $\text{HR}@k^*$ is the top- k HR of the model trained on perturbed data without any defense.

In the context of adversarial attacks, numerous studies [143, 173–176] measure defense effectiveness by evaluating the change in recommendation metrics of the target item, defined as

$$\Delta T\text{-}M = |T\text{-}M' - T\text{-}M|, \quad (24)$$

where $T\text{-}M$ denotes the recommendation metric for the target item. In practice, it is common to choose unpopular items as target items during evaluation. As a result, the value of $T\text{-}M$ in the absence of attacks is typically close to 0.0. Consequently, some studies [173, 174, 176] approximate $\Delta T\text{-}M$ directly using $T\text{-}M'$.

Compared to alternative approaches, offset-based metrics are straightforward to compute. However, they do not always provide an accurate measure of robustness. For example, consider two top- k recommendation lists $\hat{L}_u@k$ and $\hat{L}_v@k$ for users u and v in a system trained on clean data. Without loss of generality, assume $k = 3$, ideal recommendation lists $L_u = \{i_1, i_2, \dots, i_8\}$, $L_v = \{i_5, i_6, \dots, i_{12}\}$, and $\hat{L}_u@k = \{i_8, i_9, i_{10}\}$, $\hat{L}_v@k = \{i_1, i_5, i_6\}$. If, after perturbation, these lists change to $\hat{L}'_u@k = \{i_7, i_8, i_{10}\}$ and $\hat{L}'_v@k = \{i_1, i_3, i_6\}$, we obtain $\Delta_{\text{HR}@k} = 0$ despite noticeable shifts in recommendations. In such cases, offset-based metrics may fail to reflect robustness accurately.

5.1.2 Offset on Output. To address the challenge of accurately measuring recommender system robustness, some methods evaluate the offset on system output to quantify prediction shifts. This can be formulated as follows:

$$\Delta O = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \text{sim}(\hat{L}_u@k, \hat{L}'_u@k), \quad (25)$$

where $\hat{L}_u@k$ represents the top- k recommendation list from a system trained on clean data, $\hat{L}'_u@k$ denotes the top- k list from a system trained on perturbed data, and $\text{sim}(\cdot)$ is a similarity function that quantifies differences between the two lists.

Shriver et al. [116] propose a metric called **Top Output (TO)**, which is sensitive only to the top-ranked item for a user. This item typically has the highest likelihood of being preferred:

$$\text{TO} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathbb{I}[\hat{L}'_u@k[0] \in \hat{L}_u], \quad (26)$$

where $\mathbb{I}[\cdot]$ is an indicator function that returns 1 if the condition holds and 0 otherwise.

Oh et al. [93] employ two similarity metrics in Equation (25) to compute ΔO : **Rank-Biased Overlap (RBO)** [67] and **Jaccard Similarity** (Jaccard) [59]. RBO captures similarity in item ordering, whereas Jaccard measures the overlap in the top- k items regardless of order. Given two lists L_1 and

⁷ $\text{HR}@k$ measures the fraction of test items appearing in the top- k recommendation list relative to all test items.

⁸ $\text{NDCG}@k$ measures ranking quality by considering both item relevance and ranking position.

L_2 , these metrics are formulated as

$$\text{RBO}@k(L_1, L_2) = (1 - p) \sum_{d=1}^k p^{d-1} \frac{|L_1[0 : d] \cap L_2[0 : d]|}{d}, \quad (27)$$

where p is a tunable parameter (recommended value: 0.9), and

$$\text{Jaccard}@k(L_1, L_2) = \frac{|L_1[0 : k] \cap L_2[0 : k]|}{|L_1[0 : k] \cup L_2[0 : k]|}. \quad (28)$$

The top- k Jaccard metric is particularly valuable in industry due to its computational efficiency compared to RBO. Conversely, RBO enables a more detailed robustness analysis by focusing on full-ranked lists, offering a more comprehensive evaluation of recommender system stability. By leveraging these diverse metrics, researchers can gain deeper insights into system performance under various perturbations, facilitating the development of more robust and reliable models.

5.1.3 Certifiable Robustness. Certifiable robustness focuses on finding the robustness boundary for a given instance in the recommender systems model. Traditional methods for certifiable robustness [31, 75] can be categorized into two approaches: randomized smoothing and directly finding the worst perturbation. Randomized smoothing is a technique that smooths the input by applying random noise, aiming to find an adversarial boundary that causes the model to produce incorrect outputs. However, in the recommender systems scenario, it is difficult to smooth the input of the model due to the semantics and discreteness of the features.

Directly finding the worst perturbation involves searching for the worst perturbation that can lead to an incorrect prediction for a given input. Liu et al. [83] provide both non-robust certification and robust certification by approximately calculating the worst perturbation for the FM model. For a given FM model f , input sample $\mathbf{x}$, which includes historical interaction and other features, and the perturbation budget q , let $\mathbf{x}'$ denote the perturbed instance corresponding to $\mathbf{x}$. Recall the formulation of the FM model $f(\mathbf{x})$ in Equation (13), Liu et al. [83] formulate the prediction shift δ as

$$\begin{aligned} \delta = f(\mathbf{x}') - f(\mathbf{x}) &= \sum_{j=1}^d w_j(x_j - x'_j) + \sum_{f=1}^k \sum_{i=1}^d \sum_{j=1}^d v_{i,f} v_{j,f} x_i(x_j - x'_j) \\ &\quad + \frac{1}{2} \sum_{f=1}^k \left(\sum_{j=1}^d v_{j,f}(x_j - x'_j) \right)^2 + \frac{1}{2} \sum_{f=1}^k \sum_{j=1}^d v_{j,f}^2 (x_j - x'_j)^2. \end{aligned} \quad (29)$$

Due to space constraints, Appendix A.1 details the approximation process for maximizing the prediction shift δ to achieve robustness certification. In summary, certifiable robustness is a key metric for evaluating robustness, as it establishes robustness bounds for individual instances.

Additionally, numerous publicly available toolkits assist researchers in evaluating the robustness of recommender systems. For example, RecBole [188] provides a comprehensive collection of up-to-date recommender system implementations, while RecAD [133] offers a suite of existing attack and defense methods for evaluating robustness.

5.2 Common Datasets

In this survey, we have analyzed the evaluation datasets used in papers on recommender systems' robustness and found that over 50 common datasets have been utilized. However, nearly 50% of these datasets appear in only one paper, and merely 20% have been used in more than three papers.

Here, we introduce the statistics of the 20% most frequently used datasets in Table 3. The primary datasets include MovieLens, Amazon, LastFM, Netflix, Yelp, and MIND. Due to space limitations, detailed descriptions of these datasets are provided in Appendix A.2. During our statistical analysis,

Table 3. Common Datasets

Dataset	# User	# Item	# Interaction	Publication
MovieLens-100K [52]	943	1682	100,000	[4, 17, 30, 39, 44, 54, 77, 116, 127, 140, 142, 143, 149, 157, 158, 160, 191, 193, 195]
MovieLens-1M	6040	3900	1,000,209	[11, 39, 77, 118, 125, 131, 142, 143, 148, 149, 159, 163–165, 167, 171, 191]
MovieLens-20M	138,493	27,278	20,000,263	[22, 37, 113, 167]
Amazon-Beauty [92]	1,210,271	249,274	2,023,070	[18, 22, 25, 84, 115, 167, 190]
Amazon-Book	8,026,324	2,330,066	22,507,155	[23, 125, 135, 137, 145, 151, 156, 159]
LastFM [19]	980	1000	1,293,103	[20, 37, 63, 64, 93, 135, 149, 167, 190]
Netflix	480,189	17,770	100,480,507	[11, 113, 142, 147, 148, 160, 169, 191]
Yelp	1,987,897	150,346	6,990,280	[23, 25, 37, 44, 53, 54, 63, 78, 83, 84, 101, 115, 125, 136, 137, 143, 145, 147, 151, 156, 159, 180, 189, 190]
MIND [144]	1,000,000	161,013	24,155,470	[64, 173–177]

we found that the datasets used across many papers are inconsistent. In addition, some papers sample subsets from large datasets for testing, which makes it difficult to replicate experiments and can lead to unfair comparisons between methods in later studies. We encourage researchers to use more widely adopted datasets and to take a careful and rigorous approach when sampling from large datasets. Establishing standardized benchmarks would further support the field's progress.

5.3 Comparative Evaluation of Robustness Methods

In this section, we experimentally evaluate representative robustness methods across different categories. Some methods are inherently dependent on specific models or scenarios, making direct comparisons with other categories infeasible. For instance, certifiable robustness methods [83] are restricted to FM models and, therefore, are excluded from our comparative analysis.

5.3.1 Experimental Setup. Due to space constraints, we briefly introduce the experimental setup here. More detailed settings for robustness against malicious attacks can be found in [143, 173, 174], while details on robustness against natural noise are available in [137, 177].

We conduct experiments on the **MIND** dataset [144], which includes item side information. Following [173, 174], we sample users and filter out those with fewer than five interactions. To ensure comparability across model types, we adopt two representative backbones: **SASRec** [65] (sequential) and **MF** [70] (CF). For robustness evaluation, we implement the **Bandwagon Attack** [89] and simulate 10% **random misclicks** as natural noise. We use NDCG@20 to assess performance, T-NDCG@20 for attack defense, and further evaluate robustness via metric offset (Δ NDCG@20, Δ T-NDCG@20) and output offset (TO@50).

5.3.2 Robustness Methods against Malicious Attacks. Table 4 presents the performance of various robustness methods against malicious attacks. In the CF setting:

- **Adversarial training** on parameters not only reduces attack success rates but also enhances recommendation performance, aligning with the theorem provided by PamaCF [176].
- **Adversarial training on parameters better preserves output stability.** In contrast, fraudster detection or adversarial training on user profiles inherently alters user profiles by adding or removing users, leading to greater output fluctuations.

In the Sequential Recommendation setting:

- **Fraudster detection methods** show better performance in mitigating attacks, while traditional adversarial training methods (e.g., APR) tend to significantly degrade recommendation quality. This may be due to the fact that parameter perturbations are more likely to disrupt the reasoning process in sequential models, resulting in inaccurate recommendations.

Table 4. Performance Comparison Against Malicious Attacks

Category	Method	NDCG@20(↑)	NDCG@20'(↑)	ΔNDCG@20(↓)	ΔT-NDCG@20(↓)	T0@50(↑)	Time/epoch(↓)
Backbone	MF ¹	0.69±0.01	0.67±0.01	2.90±1.02	0.055±0.005	70.59±2.62	20.28±2.39
Fraudster Detection	+GraphRfi [180]	0.67±0.00	0.65±0.01	2.99±1.25	0.050±0.005	61.03±1.90	47.50±10.32
	+LLM4Dec [175]	0.68±0.00	0.68±0.01	0.32±0.91	<u>0.025±0.003</u>	65.10±1.71	-
Adversarial Training	+APR [53]	0.70±0.01	0.71±0.01	1.43±1.62	0.026±0.003	90.22±2.91	26.19±3.03
	+APT [143]	0.65±0.02	0.63±0.02	3.08±2.96	0.046±0.002	66.52±5.07	-
	+PamaCF [176]	0.71±0.00	0.73±0.00	2.82±0.27	0.006±0.001	96.31±1.66	26.78±2.96
Backbone	SASrec	6.50±0.33	6.34±0.50	2.46±1.51	0.083±0.001	60.59±2.02	197.04±4.66
Fraudster Detection	+GraphRfi [180]	6.35±0.39	6.29±0.21	0.94±1.35	0.067±0.001	69.72±3.50	310.57±26.72
	+LLM4Dec [175]	6.39±0.27	6.31±0.16	1.25±0.90	0.060±0.002	81.27±4.63	-
	+LoRec [174]	6.52±0.20	6.72±0.13	3.07±0.68	0.003±0.001	<u>76.26±3.91</u>	260.86±13.01
Adversarial Training	+APR [53]	6.14±0.39	6.09±0.33	0.81±1.68	0.088±0.001	70.34±2.10	245.82±15.18
	+ADVTrain [167]	6.33±0.41	6.36±0.38	<u>0.47±1.31</u>	0.104±0.001	71.01±3.66	226.27±6.02

¹When the number of epochs in the MF model is too high, the T-NDCG@20 scores across different defense methods become excessively low, making comparisons difficult [143]. To ensure a fair comparison, we follow the epoch settings as proposed in [143].

Table 5. Performance Comparison against Natural Noise

Category	Method	NDCG@20(↑)	NDCG@20'(↑)	ΔNDCG@20(↓)	T0@50(↑)	Time/epoch(↓)
Backbone	MF	4.57±0.02	3.55±0.05	22.32±1.05	64.70±2.30	20.36±1.98
Regularization	+R1 [169]	4.01±0.03	3.29±0.02	17.96±4.03	70.39±2.12	21.20±1.88
Purification	+ADT [137]	4.90±0.12	4.04±0.09	17.55±2.05	63.95±6.97	22.89±1.88
	+BOD [141]	5.16±0.10	4.60±0.12	10.85±1.59	87.47±3.27	48.59±1.02
	+PLD [177]	5.24±0.07	4.76±0.10	9.16±1.10	<u>86.85±1.59</u>	21.95±1.81
Self-supervised Learning	+DeCA [140]	5.19±0.06	4.21±0.09	18.88±2.09	74.66±5.80	29.32±4.17
Backbone	SASrec	6.50±0.33	5.46±0.51	16.00±2.48	25.03±5.31	197.04±4.66
Regularization	+R1 [169]	5.78±0.50	5.19±0.29	10.21±3.73	26.79±1.55	209.11±5.00
Purification	+ADT [137]	6.00±0.52	5.49±0.30	8.50±1.78	26.79±2.81	212.73±3.17
	+PLD [177]	6.60±0.23	6.08±0.28	7.88±1.45	30.78±1.39	207.59±2.88
Self-supervised Learning	+S ³ -Rec [190]	6.61±0.29	<u>6.06±0.39</u>	8.32±1.61	<u>28.60±3.76</u>	251.39±4.39

- **Leveraging LLM-based open-world knowledge** substantially enhances defense effectiveness. Both LLM4Dec and LoRec markedly reduce attack success rates while offering greater output stability compared to other methods.

In summary, in CF-based recommender systems, parameter-level adversarial training is a preferred defense strategy due to its dual benefits of robustness and performance enhancement [176]. However, detection methods must accurately identify fraudsters while minimizing modifications to user profiles to maintain output stability. In Sequential Recommendation, Fraudster Detection approaches are more suitable for defense. Furthermore, integrating external knowledge in the detection phase has the potential to enhance detection accuracy and overall robustness.

5.3.3 Robustness Methods against Natural Noise. Table 5 presents the performance of various robustness methods against natural noise. Our key observations are:

- **Regularization** is not universally suitable for all types of recommender systems. While improving output stability to some extent, they significantly degrade recommendation performance.
- **Purification and Self-Supervised Learning (SSL) methods** both effectively enhance recommendation performance. Compared to SSL, Purification exhibits superior output stability, particularly PLD. PLD modifies the sampling strategy for positive samples during training, which increases the likelihood of selecting clean samples rather than directly discarding potential noise.

In summary, both Purification and SSL serve as strong defenses against natural noise. Future research should explore hybrid strategies that combine SSL with purification techniques, leveraging their complementary strengths to enhance both robustness and recommendation performance.

6 Robustness in Various Recommendation Scenarios

This section explores the considerations of robustness in recommender systems from two perspectives: the nature of the recommendation task and the context of the application. In the first subsection, we delve into the four main types of recommendation tasks—content-based, CF-based, sequential, and hybrid. Challenges in each of these tasks primarily arise from their distinct operational methodologies and their reliance on various data types.

In the second subsection, we shift our focus to the application contexts of these recommender systems. Specifically, we discuss e-commerce, media, news, and social recommender systems. We aim to highlight how robustness requirements vary based on their application context.

6.1 From the Perspective of Recommendation Tasks

Content-based Recommender Systems: In these systems [86], robustness depends heavily on the quality of item metadata, such as tags, descriptions, and other side information. They recommend items similar to those the user has liked before, making them especially vulnerable to errors or noise in the item metadata. Such issues can significantly affect the system's ability to measure item similarity accurately, leading to poor recommendations. Therefore, a key research goal in these systems is to design algorithms that improve the reliability of side information (Section 3.2.3).

CF-based Recommender Systems: CF-based recommender systems generate recommendations based on the premise that users with similar behaviors tend to share similar preferences. Since these systems rely on user behaviors, they are vulnerable to interference from malicious users, particularly shilling attacks [29, 122], which can distort user behavior patterns, leading to biased and inaccurate recommendations [71]. Given these vulnerabilities, research efforts in CF-based systems primarily focus on detecting and mitigating such attacks (Section 3). Specifically, researchers are developing methods to identify these attacks and devising training methods that bolster the system's defenses, ensuring recommendation accuracy and reliability [175].

Sequential Recommender Systems: Sequential recommender systems generate recommendations based on users' historical interaction sequences [100]. Consequently, they are highly sensitive to temporal fluctuations, such as evolving user preferences, seasonal variations, and emerging trends [167]. The primary challenge in these systems is to maintain reliable recommendations despite such changes in user behavior. To address this, researchers are focusing on designing models that are more resilient to deviations in users' historical behavior (Sections 4 and 3.2.4).

Hybrid Recommender Systems: Hybrid recommender systems integrate multiple recommendation strategies to leverage their respective strengths [14]. Consequently, they face a diverse set of robustness challenges, each stemming from the methods they incorporate. Researchers in this domain aim to develop an optimal combination of these techniques, ensuring that the strengths of one approach compensate for the weaknesses of another. The goal is to create a robust framework that delivers consistently accurate recommendations across varying scenarios.

6.2 From the Perspective of Applications

E-commerce Recommender Systems: E-commerce recommender systems are a key feature of platforms like Amazon and eBay, helping customers discover products more easily [117]. These systems typically rely on historical browsing and purchase data, using both user-item interactions and item-side information. Despite their effectiveness, they face several robustness challenges. One significant issue is the presence of natural noise in user data, such as unintentional clicks or purchases that may not accurately reflect genuine user preferences [160]. Additionally, attackers can manipulate item-side information to promote specific products or fabricate user profiles to artificially boost a product's exposure [32]. In an e-commerce context, ensuring recommender system robustness requires a multifaceted approach. Beyond the strategies outlined in Sections 3

Table 6. Relationship between Robustness and Other Key Properties in Recommender Systems

Property	Relationship with Robustness
Accuracy	Often negatively correlated (↓); robustness may reduce accuracy on clean data
Interpretability	Positively correlated (↑); interpretable models facilitate adversarial detection
Privacy	Positively correlated (↑); privacy-preserving techniques often enhance robustness
Fairness	Positively correlated (↑); robust models can mitigate bias, but fairness operates under different constraints

and 4, real-world applications demand that these systems adapt to the dynamic nature of product trends, handle extensive product catalogs, and maintain the long-term relevance of products. These factors collectively introduce new challenges in sustaining robustness.

Media Recommender Systems: Media recommender systems are widely used on streaming platforms like Netflix and TikTok. They suggest content—such as movies or songs—based on a user’s watch or listen history and stated preferences [47]. Like e-commerce systems, short-media recommendations face robustness issues, including accidental likes, misleading video tags, and orchestrated efforts to inflate likes or views (Sections 4 and 3). These challenges are further intensified by the rapid shift in trends and the short lifespan of content. Conversely, long-media recommendations—such as those for movies and music—better capture user preferences through indicators like viewing or listening duration, which mitigates the impact of natural noise. However, they are still vulnerable to fake user interactions [122], and the manipulation of item information, such as fake tags, remains common due to the promotional nature of media content [32] (Section 3).

News Recommender Systems: News recommender systems, commonly used on news websites and apps, suggest articles based on a user’s reading history and stated interests [66]. These systems face unique challenges due to the short lifespan of news content and their core role in spreading information, which makes them attractive targets for attackers aiming to spread misinformation. In this context, the primary concern is not just the presence of noise but whether attackers have manipulated news content to introduce false information, as discussed in Section 3.2.3.

Social Recommender Systems: Social recommender systems, common on platforms such as Facebook and X (formerly known as Twitter), suggest connections, pages, or groups based on user interactions and existing social ties [155]. A distinguishing characteristic of these systems is that every participant functions as both a user and an item. This dual role heightens the significance of side information, including user-generated content and social interactions, such as following activities [35]. Given the prevalence of biased content and artificial followers on social platforms, ensuring robustness is paramount. A key challenge is detecting and mitigating the influence of harmful content, fake users, and social bots. To address these challenges, researchers are developing methods for detecting fraudulent activity and safeguarding platform authenticity, as outlined in Section 3.1.

7 Relationship with Other Trustworthy Properties

Robustness is a key property of trustworthy recommender systems. In this section, we examine how robustness interacts with other important aspects of trustworthiness. These four aspects are accuracy, interpretability, privacy, and fairness. We explore the possible tradeoffs and synergies that may occur when improving robustness alongside these goals, as illustrated in Table 6.

7.1 Robustness vs. Accuracy

Several studies have shown that improving model robustness often reduces accuracy, especially on clean data [109, 134]. This tradeoff has also been theoretically established in specific scenarios [130, 162]. The tradeoff often arises due to the limitations of defense methods, such as the difficulty in identifying adversarial examples that subtly alter label semantics. Another reason is the model’s struggle to learn an optimal decision boundary after adversarial perturbations are introduced.

Within recommender systems, this tradeoff remains evident. When facing malicious data—whether due to natural noise or attacks—researchers often introduce assumptions to build more robust models. Some approaches emphasize distinguishing between clean and adversarial data during training, aiming to detect and minimize the impact of “adversarial-like” data by reducing their weight in training [44, 137]. While these methods effectively mitigate the effects of noise or attacks, they often lead to fundamental divergences in feature representations between optimal standard and robust models [130], thereby reducing recommendation accuracy on clean data.

In real-world scenarios, reducing the tradeoff between robustness and accuracy is critically important. Since most recommender systems are designed to serve platform goals, significant computational resources are invested in improving robustness. Notably, in certain settings such as CF, [176] theoretically demonstrate that adversarial training can improve both robustness and recommendation performance. This finding offers new insights into mitigating the tradeoff between accuracy and robustness in recommender systems. However, how to generalize these benefits to other recommendation paradigms remains an open question for future research.

7.2 Robustness vs. Interpretability

Interpretability in machine learning and deep learning refers to how clearly a model’s decision-making process can be understood, either locally or globally [182]. A highly interpretable model not only reveals the basis for its decisions but also explains the key factors influencing its predictions. Recent studies highlight a strong correlation between them [79, 128], suggesting that understanding how a model behaves on adversarial samples can help improve robustness.

In recommender systems, there is a growing emphasis on interpretability [111, 181]. For instance, [181] formulates recommendation as neuro-symbolic reasoning, where symbolic reasoning not only explains the model’s logic but also helps identify noise or attacks by tracing the reasoning behind incorrect recommendations. In real-world systems, robustness and interpretability often support each other. Users are more likely to trust recommendations when they are both robust and interpretable, fostering greater economic value. From a technical standpoint, these objectives reinforce each other, making their joint pursuit beneficial.

7.3 Robustness vs. Privacy

Privacy refers to protecting individuals’ or organizations’ information, actions, and identities from unauthorized access, use, or disclosure [36]. There is a complex interplay between privacy and robustness, with each potentially enhancing the other. Prior research suggests that privacy algorithms can inherently provide robustness, and some general methods exist for converting robust algorithms into privacy-preserving ones [41]. A widely used approach to protecting privacy is **Differential Privacy (DP)**, which reduces the risk of exposing individual data by adding noise to data or model parameters [1, 40]. Interestingly, incorporating DP has been shown to enhance model robustness, making them less sensitive to small perturbations [73, 91].

In recommender systems, DP is especially relevant in federated learning scenarios. When users send their data to central servers for gradient computation, they often add perturbations to protect privacy [87, 114]. Similar perturbation techniques are applied in adversarial training for recommender systems [23, 53]. As real-world recommender systems increasingly operate under privacy regulations, ensuring both privacy and robustness through their mutual reinforcement offers a promising solution for secure recommendations.

7.4 Robustness vs. Fairness

Fairness in machine learning refers to generating impartial and unbiased predictions across different groups or individuals [96, 139]. Many approaches define fairness as ensuring model predictions

remain unchanged under variations in sensitive attributes (e.g., gender) [69, 96]. This idea resembles adversarial training, where small perturbations in input data should not alter predictions [166]. However, a key distinction remains: fairness focuses on changes to specific attributes regardless of their size, whereas robustness enforces that changes remain small to preserve original labels. Recent studies suggest that enhancing robustness can indirectly improve fairness [98].

In recommender systems, fairness on the user side closely follows traditional definitions, ensuring that recommendations remain consistent when users' sensitive attributes change [139]. However, robustness in recommender systems mainly aims to reduce the impact of noisy interactions or malicious behavior, which involves a different set of concerns. That said, in real-world scenarios, the scope of noise might extend beyond just user behaviors, and fairness issues may begin to overlap with robustness objectives. This possible convergence points to a promising direction for future work on using robustness techniques to support fairness in recommender systems.

8 Open Issues and Future Directions

As recommender systems evolve, they present numerous challenges to researchers seeking to develop robust models that maintain their performance under various conditions and perturbations. This section explores open issues and future directions in the field of robust recommender systems.

8.1 Mitigating Gap between Defense Assumption and Attack Goal

A key challenge in robust recommender systems is the mismatch between defense assumptions and the actual goals of attacks. Attacks, especially shilling attacks, often aim to promote or diminish product exposure rather than merely undermining recommendation performance. However, many existing defense methods are built on the assumption that attackers primarily seek to disrupt the functionality of recommender systems. For instance, He et al. [53] incorporate adversarial perturbations into model parameters during the training phase. Yet, these perturbations often fail to match real-world attack patterns, meaning some adversarial examples optimized during training might be ineffective for defense. In another method, Wu et al. [143] propose adding empirical risk-minimizing users to defend fraudsters who are treated as a threat to recommendation quality.

Such approaches often struggle to address attacks driven by different motivations, leading to fragmented and less effective defenses. To advance the robustness of recommender systems, future research should aim to bridge this gap by aligning defense assumptions more closely with real-world attack goals. Promising directions include: (1) imposing more rigorous constraints on adversarial samples during training—potentially by adopting knowledge-enhanced adversarial perturbations or similar techniques that infuse prior knowledge into perturbation generation. And (2) considering new adversarial training paradigms could be valuable. For instance, shifting from the established “min-max” framework to a “min-min” perspective [143] might pave the way for a more specific defense against attacks like shilling.

8.2 Improving Generalization of Defense Methods

The generalization of defense methods in recommender systems presents a considerable challenge. Many existing strategies focus on specific assumptions or model constraints. For example, Liu et al. [83] is tailored exclusively for models based on the FM, while Cao et al. [18] is designed for models grounded in RL. Moreover, numerous pre-processing detection techniques hinge on predefined features for detecting malicious users [11, 74], and some resort to specific attack methods for generating supervised data [158, 171]. These model- or attack-specific designs limit the applicability of defenses across different scenarios, making it difficult to handle adversarial threats in large-scale, real-world systems.

Given these limitations, there is a growing need to develop defense methods that are scalable and generalizable across diverse models and environments. Future research should focus on shared characteristics of poisoned data and general patterns in model training or loss functions, rather than overly tailoring defenses to specific models, datasets, or attack types. Promising directions include: (1) The development of feature-free detection methods, ensuring they neither depend on pre-selected features nor limit dataset adaptability. (2) In-process detection methodologies that can discern patterns exhibited by malicious users during the model's training phase. (3) Leveraging external knowledge to enhance generalization, such as incorporating LLM-based semantic understanding.

8.3 Standard Evaluation and Benchmark

A main challenge in robust recommender system research is the absence of consistent evaluation methods and standardized benchmark datasets. As discussed in Section 5.1, most methods compute the offset on metrics to indirectly show the robustness of the recommender system. However, the offset on the metrics sometimes does not effectively measure the robustness. Furthermore, as pointed out in Section 5.2, more than 50 datasets are commonly used, but only about 20% of them are referenced in more than three studies. Notably, only three datasets are used in over 10 papers.

8.4 Trustworthy Recommender Systems

As discussed in Section 7, the relationship between robustness and other trustworthiness properties—such as interpretability, privacy, and fairness—is complex. These properties can either reinforce one another or conflict, especially in relation to accuracy. One of the main challenges in current research is managing these tradeoffs with accuracy. Several studies have shown that improving a model's robustness [130], interpretability [10], privacy [104], or fairness [33] often leads to reduced accuracy. While some strides have been made to diminish these tradeoffs—by establishing the upper bounds of these tradeoffs in specific instances [130, 162, 176]—there remains ample scope for advancement. Future avenues of exploration include: (1) The theoretical examination of upper bounds in more intricate scenarios. (2) Investigating the interplay between various trustworthiness properties and accuracy, with the intent of deciphering their shared traits and distinctions, and thereby identifying more optimal balancing techniques.

Another pressing research challenge entails the synergistic enhancement of robustness, interpretability, privacy, and fairness. Existing literature has suggested that the enhancement techniques for these properties often share similar optimization objectives [73, 91, 98]. Moreover, certain methodologies can be adapted from one property to another [41]. Forward-looking research should revisit the formal definitions of each property, aiming to delineate unified optimization objectives or devise methods to translate strategies across properties.

8.5 LLM for Robust Recommender Systems

LLMs, like ChatGPT [124], have brought significant advancements in NLP in recent years. These models, trained on vast amounts of text data, are capable of generating human-like text, answering questions, and performing various language tasks with high accuracy. Lately, there's been a growing interest in utilizing LLMs for a range of tasks beyond their usual scope [42, 45]. Moreover, some studies indicate that LLMs can be employed to improve model robustness [26]. The broad knowledge base within LLMs can improve defense mechanisms. Future research avenues in this area could include: (1) Leveraging LLMs in adversarial training, for instance, in generating adversarial examples. (2) Using LLMs to enhance techniques for detecting fraudsters. (3) Investigating recommendation systems based on LLMs, taking advantage of their built-in robustness for delivering more reliable recommendations.

8.6 Defending Adaptive Attackers

Most existing defense methods assume that attackers are unaware of the defense mechanisms, effectively treating the attack process as fully black-box. While this assumption simplifies evaluation, it limits the ability to assess the true robustness of defense strategies. In real-world scenarios, however, the interaction between attackers and defenders is a dynamic and continuous adversarial process. Attackers actively adapt their strategies based on the current robustness of the recommender system. For instance, when detection mechanisms are deployed, attackers may adopt more stealthy behaviors to evade detection. Likewise, as adversarial training techniques become more common, attackers may shift toward more destructive or targeted strategies. Robustness can be more meaningfully evaluated in such adaptive attack settings, where the attacker deliberately responds to the defense. From the defender's perspective, it is essential to develop dynamic defense strategies that can continuously learn and respond to evolving attacks. Promising directions include RL-based defenses, adversarial game-theoretic frameworks, and self-evolving training methods that proactively counter adaptive attacks.

9 Conclusion

In this survey, we offer a comprehensive review of seminal contributions in the field of robust recommender systems. We propose a taxonomy to systematically organize the vast array of publications in this domain. Specifically, we examine the robustness of recommender systems along two primary dimensions: malicious attacks and natural noise. Regarding techniques, we delve into state-of-the-art methods for building robust recommender systems, including methods tailored for malicious attacks—fraudster detection, adversarial training, and certifiable robust training—as well as methods designed to mitigate natural noise—regularization, purification, and SSL. Additionally, we discuss evaluation metrics and prominent datasets, shedding light on how robustness in recommender systems is assessed. Our analysis explores the nuanced dimensions of robustness across various scenarios and its interplay with other trustworthy properties, such as accuracy, interpretability, privacy, and fairness. We also highlight open challenges and envision potential future directions in robust recommender systems research. Our goal with this survey is to equip researchers with a thorough understanding of the key aspects of robust recommender systems, elucidate significant advancements, and inspire further exploration of this critical area.

References

- [1] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep learning with differential privacy. In *SIGSAC*. 308–318.
- [2] Nor Aniza Abdullah, Rasheed Abubakar Rasheed, Mohd Hairul Nizam Md Nasir, and Md Mujibur Rahman. 2021. Eliciting auxiliary information for cold start user recommendation: A survey. *Appl. Sci.* 11, 20 (2021), 9608.
- [3] M. Mehdi Afsar, Trafford Crump, and Behrouz Far. 2022. Reinforcement learning based recommender systems: A survey. *ACM Comput. Surv.* 55, 7 (2022), 1–38.
- [4] Mehmet Aktukmak, Yasin Yilmaz, and Ismail Uysal. 2019. Quick and accurate attack detection in recommender systems through user attributes. In *RecSys*. 348–352.
- [5] Bushra Alhijawi, Arafat Awajan, and Salam Fraihat. 2022. Survey on the objectives of recommender systems: Measures, solutions, evaluation methodology, and new perspectives. *ACM Comput. Surv.* 55, 5 (2022), 1–38.
- [6] Vito Walter Anelli, Yashar Deldjoo, Tommaso Di Noia, Daniele Malitesta, and Felice Antonio Merra. 2021. A study of defensive methods to protect visual recommendation against adversarial manipulation of images. In *SIGIR*. 1094–1103.
- [7] Vito Walter Anelli, Yashar Deldjoo, Tommaso Di Noia, and Felice Antonio Merra. 2020. Adversarial learning for recommendation: Applications for security and generative tasks—concept to code. In *RecSys*. 738–741.
- [8] Vito Walter Anelli, Yashar Deldjoo, Tommaso Di Noia, and Felice Antonio Merra. 2021. Adversarial recommender systems: Attack, defense, and advances. In *Recommender Systems Handbook*. Springer, 335–379.
- [9] Linas Baltrunas, Bernd Ludwig, and Francesco Ricci. 2011. Matrix factorization techniques for context aware recommendation. In *RecSys*. 301–304.

- [10] Andrew Bell, Ian Solano-Kamaiko, Oded Nov, and Julia Stoyanovich. 2022. It's just not that simple: An empirical study of the accuracy-explainability trade-off in machine learning for public policy. In *FACCT*. 248–266.
- [11] Alex Beutel, Kenton Murray, Christos Faloutsos, and Alexander J. Smola. 2014. Cobafi: Collaborative Bayesian filtering. In *TheWebConf*. 97–108.
- [12] Zhi Bian, Shaojun Zhou, Hao Fu, Qihong Yang, Zhenqi Sun, Junjie Tang, Guiquan Liu, Kaikui Liu, and Xiaolong Li. 2021. Denoising user-aware memory network for recommendation. In *RecSys*. 400–410.
- [13] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. 2000. LOF: Identifying density-based local outliers. In *SIGMOD*. 93–104.
- [14] Robin Burke. 2002. Hybrid recommender systems: Survey and experiments. *User Model. User-Adapt. Interact.* 12, 4 (2002), 331–370.
- [15] Robin Burke, Bamshad Mobasher, Chad Williams, and Runa Bhaumik. 2006. Classification features for attack detection in collaborative recommender systems. In *SIGKDD*. 542–547.
- [16] Robin Burke, Michael P. O'Mahony, and Neil J. Hurley. 2015. Robust collaborative recommendation. In *Recommender Systems Handbook*, Francesco Ricci, Lior Rokach, Bracha Shapira, and Paul B. Kantor (Eds.). Springer, 961–995.
- [17] Jie Cao, Zhiang Wu, Bo Mao, and Yanchun Zhang. 2013. Shilling attack detection utilizing semi-supervised learning method for collaborative recommender system. *World Wide Web* 16, 5 (2013), 729–748.
- [18] Yuanjiang Cao, Xiaocong Chen, Lina Yao, Xianzhi Wang, and Wei Emma Zhang. 2020. Adversarial attacks and detection on reinforcement learning-based interactive recommender systems. In *SIGIR*. 1669–1672.
- [19] O. Celma. 2010. *Music Recommendation and Discovery in the Long Tail*. Springer.
- [20] Huiyuan Chen and Jing Li. 2019. Adversarial tensor factorization for context-aware recommendation. In *RecSys*. 363–367.
- [21] Huiyuan Chen, Xiaoting Li, Vivian Lai, Chin-Chia Michael Yeh, Yujie Fan, Yan Zheng, Mahashweta Das, and Hao Yang. 2023. Adversarial collaborative filtering for free. In *RecSys*. 245–255.
- [22] Huiyuan Chen, Yusan Lin, Menghai Pan, Lan Wang, Chin-Chia Michael Yeh, Xiaoting Li, Yan Zheng, Fei Wang, and Hao Yang. 2022. Denoising self-attentive sequential recommendation. In *RecSys*. 92–101.
- [23] Huiyuan Chen, Kaixiong Zhou, Kwei-Herng Lai, Xia Hu, Fei Wang, and Hao Yang. 2022. Adversarial graph perturbations for recommendations at scale. In *SIGIR*. 1854–1858.
- [24] Jiawei Chen, Hande Dong, Xiang Wang, Fuli Feng, Meng Wang, and Xiangnan He. 2023. Bias and debias in recommender system: A survey and future directions. *ACM Trans. Inf. Syst.* 41, 3 (2023), 1–39.
- [25] Yongjun Chen, Zhiwei Liu, Jia Li, Julian McAuley, and Caiming Xiong. 2022. Intent contrastive learning for sequential recommendation. In *TheWebConf*. 2172–2182.
- [26] Zheng Chen, Ziyang Jiang, Fan Yang, Eunah Cho, Xing Fan, Xiaojiang Huang, Yanbin Lu, and Aram Galstyan. 2023. Graph meets LLM: A novel approach to collaborative filtering for robust conversational understanding. In *EMNLP: Industry Track*. 811–819.
- [27] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishi Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ipsir, et al. 2016. Wide & deep learning for recommender systems. In *RecSys*. 7–10.
- [28] Paul-Alexandru Chirita, Wolfgang Nejdl, and Cristian Zamfir. 2005. Preventing shilling attacks in online recommender systems. In *WIDM*. 67–74.
- [29] Konstantina Christakopoulou and Arindam Banerjee. 2019. Adversarial attacks on an oblivious recommender. In *RecSys*. 322–330.
- [30] Chen-Yao Chung, Ping-Yu Hsu, and Shih-Hsiang Huang. 2013. βP : A novel approach to filter out malicious rating profiles from recommender systems. *Decis. Support Syst.* 55, 1 (2013), 314–325.
- [31] Jeremy Cohen, Elan Rosenfeld, and Zico Kolter. 2019. Certified adversarial robustness via randomized smoothing. In *ICML*. PMLR, 1310–1320.
- [32] Rami Cohen, Oren Sar Shalom, Dietmar Jannach, and Amihoud Amir. 2021. A black-box attack model for visually-aware recommender systems. In *WSDM*. 94–102.
- [33] A. Feder Cooper, Ellen Abrams, and Na Na. 2021. Emergent unfairness in algorithmic fairness-accuracy trade-off research. In *AAAI*. 46–54.
- [34] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for YouTube recommendations. In *RecSys*. 191–198.
- [35] Stefano Cresci. 2020. A decade of social bot detection. *Commun. ACM* 63, 10 (2020), 72–83.
- [36] Emiliano De Cristofaro. 2020. An overview of privacy in machine learning. *arXiv preprint arXiv:2005.08679* (2020).
- [37] Yashar Deldjoo, Tommaso Di Noia, Eugenio Di Sciascio, and Felice Antonio Merra. 2020. How dataset characteristics affect the robustness of collaborative recommendation models. In *SIGIR*. 951–960.
- [38] Yashar Deldjoo, Tommaso Di Noia, and Felice Antonio Merra. 2021. A survey on adversarial recommender systems: From attack/defense strategies to generative adversarial networks. *ACM Comput. Surv.* 54, 2 (2021), 1–38.

- [39] Yali Du, Meng Fang, Jinfeng Yi, Chang Xu, Jun Cheng, and Dacheng Tao. 2018. Enhancing the robustness of neural collaborative filtering systems under malicious attacks. *IEEE Trans. Multimedia* 21, 3 (2018), 555–565.
- [40] Cynthia Dwork. 2006. Differential privacy. In *ICALP*. Springer, 1–12.
- [41] Cynthia Dwork and Jing Lei. 2009. Differential privacy and robust statistics. In *STOC*. 371–380.
- [42] Zihuai Zhao, Wenqi Fan, Jiatong Li, Yunqing Liu, Xiaowei Mei, Yiqi Wang, Zhen Wen, Fei Wang, Xiangyu Zhao, Jiliang Tang, and Qing Li. 2024. Recommender systems in the era of large language models (LLMs). *IEEE TKDE* 36, 11 (2024), 6889–6907.
- [43] Alhussein Fawzi, Seyed-Mohsen Moosavi-Dezfooli, and Pascal Frossard. 2017. The robustness of deep networks: A geometrical perspective. *IEEE Signal Process. Mag.* 34, 6 (2017), 50–62.
- [44] Yunjun Gao, Yuntao Du, Yujia Hu, Lu Chen, Xinjun Zhu, Ziquan Fang, and Baihua Zheng. 2022. Self-guided learning to denoise for robust recommendation. In *SIGIR*. 1412–1422.
- [45] Yunfan Gao, Tao Sheng, Youlin Xiang, Yun Xiong, Haofen Wang, and Jiawei Zhang. 2023. Chat-REC: Towards interactive and explainable LLMs-augmented recommender system. *arXiv preprint arXiv:2303.14524* (2023).
- [46] Yingqiang Ge, Shuchang Liu, Zuohui Fu, Juntao Tan, Zelong Li, Shuyuan Xu, Yunqi Li, Yikun Xian, and Yongfeng Zhang. 2024. A survey on trustworthy recommender systems. *ACM Trans. Recomm. Syst.* 3, 2 (2024).
- [47] Carlos A. Gomez-Uribe and Neil Hunt. 2015. The Netflix recommender system: Algorithms, business value, and innovation. *ACM Trans. Manag. Inf. Syst.* 6, 4 (2015), 1–19.
- [48] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and harnessing adversarial examples. In *ICLR*.
- [49] Jyotirmoy Gope and Sanjay Kumar Jain. 2017. A survey on solving cold start problem in recommender systems. In *ICCCA*. IEEE, 133–138.
- [50] Asela Gunawardana, Guy Shani, and Sivan Yogev. 2012. Evaluating recommender systems. In *Recommender Systems Handbook*. Springer, 547–601.
- [51] Ihsan Gunes, Cihan Kaleli, Alper Bilge, and Huseyin Polat. 2014. Shilling attacks against recommender systems: A comprehensive survey. *Artif. Intell. Rev.* 42, 4 (2014), 767–799.
- [52] F. Maxwell Harper and Joseph A. Konstan. 2015. The movielens datasets: History and context. *ACM Trans. Interact. Intell. Syst.* 5, 4 (2015), 1–19.
- [53] Xiangnan He, Zhankui He, Xiaoyu Du, and Tat-Seng Chua. 2018. Adversarial personalized ranking for recommendation. In *SIGIR*. 355–364.
- [54] Zhuangzhuang He, Yifan Wang, Yonghui Yang, Peijie Sun, Le Wu, Haoyue Bai, Jinqi Gong, Richang Hong, and Min Zhang. 2024. Double correction framework for denoising recommendation. In *SIGKDD*. 1062–1072.
- [55] Dan Hendrycks, Mantas Mazeika, Saurav Kadavath, and Dawn Song. 2019. Using self-supervised learning can improve model robustness and uncertainty. In *NeurIPS*, Vol. 32.
- [56] Bryan Hooi, Hyun Ah Song, Alex Beutel, Neil Shah, Kijung Shin, and Christos Faloutsos. 2016. Fraudar: Bounding graph fraud in the face of camouflage. In *SIGKDD*. 895–904.
- [57] Renjun Hu, Charu C. Aggarwal, Shuai Ma, and Jinpeng Huai. 2016. An embedding approach to anomaly detection. In *ICDE*. IEEE, 385–396.
- [58] Peter J. Huber. 2011. Robust Statistics. In *Int. Encycl. Stat. Sci.* Miodrag Lovric (Ed.). Springer, 1248–1251.
- [59] Paul Jaccard. 1912. The distribution of the flora in the alpine zone. 1. *New Phytol.* 11, 2 (1912), 37–50.
- [60] Matthew Jagielski, Alina Oprea, Battista Biggio, Chang Liu, Cristina Nita-Rotaru, and Bo Li. 2018. Manipulating machine learning: Poisoning attacks and countermeasures for regression learning. In *SP*. IEEE, 19–35.
- [61] Ashish Jaiswal, Ashwin Ramesh Babu, Mohammad Zaki Zadeh, Debapriya Banerjee, and Fillia Makedon. 2020. A survey on contrastive self-supervised learning. *Technologies* 9, 1 (2020), 2.
- [62] Dietmar Jannach, Markus Zanker, Alexander Felfernig, and Gerhard Friedrich. 2010. *Recommender Systems: An Introduction*. Cambridge University Press, -.
- [63] Yangqin Jiang, Chao Huang, and Lianghao Huang. 2023. Adaptive graph contrastive learning for recommendation. In *SIGKDD*. 4252–4261.
- [64] Yangqin Jiang, Yuhao Yang, Lianghao Xia, and Chao Huang. 2024. Diffkg: Knowledge graph diffusion model for recommendation. In *WSDM*. 313–321.
- [65] Wang-Cheng Kang and Julian J. McAuley. 2018. Self-attentive sequential recommendation. In *ICDM*. 197–206.
- [66] Mozghan Karimi, Dietmar Jannach, and Michael Jugovac. 2018. News recommender systems—Survey and roads ahead. *Inf. Process. Manag.* 54, 6 (2018), 1203–1227.
- [67] Maurice George Kendall. 1948. *Rank Correlation Methods*. Griffin.
- [68] Hyunjik Kim, George Papamakarios, and Andriy Mnih. 2021. The Lipschitz constant of self-attention. In *ICML*. PMLR, 5562–5571.
- [69] Michael P. Kim, Amirata Ghorbani, and James Zou. 2019. Multiaccuracy: Black-box post-processing for fairness in classification. In *AAAI*. 247–254.

- [99] Yuqi Qin, Pengfei Wang, and Chenliang Li. 2021. The world is binary: Contrastive learning for denoising next basket recommendation. In *SIGIR*. 859–868.
- [100] Massimo Quadrana, Paolo Cremonesi, and Dietmar Jannach. 2018. Sequence-aware recommender systems. *ACM Comput. Surv.* 51, 4 (2018), 1–36.
- [101] Yuhan Quan, Jingtao Ding, Chen Gao, Lingling Yi, Depeng Jin, and Yong Li. 2023. Robust preference-guided denoising for graph based social recommendation. In *TheWebConf*. 1097–1108.
- [102] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *UAI*. 452–461.
- [103] Steffen Rendle and Lars Schmidt-Thieme. 2010. Pairwise interaction tensor factorization for personalized tag recommendation. In *WSDM*. 81–90.
- [104] Shahbaz Rezaei, Zubair Shafiq, and Xin Liu. 2023. Accuracy-privacy trade-off in deep ensemble: A membership inference perspective. In *SP*. IEEE, 364–381.
- [105] Fatemeh Rezaimehr and Chitra Dadkhah. 2021. A survey of attack detection approaches in collaborative filtering recommender systems. *Artif. Intell. Rev.* 54, 3 (2021), 2011–2066.
- [106] Emanuele Rossi, Ben Chamberlain, Fabrizio Frasca, Davide Eynard, Federico Monti, and Michael Bronstein. 2020. Temporal graph networks for deep learning on dynamic graphs. In *ICML Workshop on Graph Representation Learning*.
- [107] Masahiro Sato, Sho Takemori, Janmajay Singh, and Tomoko Ohkuma. 2020. Unbiased learning for the causal effect of recommendation. In *RscSys*. 378–387.
- [108] Madeline C. Schiappa, Yogesh S. Rawat, and Mubarak Shah. 2023. Self-supervised learning for videos: A survey. *ACM Comput. Surv.* 55, 13s (2023), 1–37.
- [109] Ludwig Schmidt, Shibani Santurkar, Dimitris Tsipras, Kunal Talwar, and Aleksander Madry. 2018. Adversarially robust generalization requires more data. In *NeurIPS*, Vol. 31.
- [110] Suvash Sedhain, Aditya Krishna Menon, Scott Sanner, and Lexing Xie. 2015. Autorec: Autoencoders meet collaborative filtering. In *TheWebConf*. 111–112.
- [111] Sungyong Seo, Jing Huang, Hao Yang, and Yan Liu. 2017. Interpretable convolutional neural networks with dual local and global attention for review rating prediction. In *RecSys*. 297–305.
- [112] Huawei Shen, Yuanhao Liu, Kaike Zhang, Qi Cao, and Xueqi Cheng. 2025. The rising safety concerns of deep recommender systems. *The Innovation* (2025), 101038.
- [113] Ilya Shenbin, Anton Alekseev, Elena Tutubalina, Valentin Malykh, and Sergey I. Nikolenko. 2020. Recvae: A new variational autoencoder for top-n recommendations with implicit feedback. In *WSDM*. 528–536.
- [114] Hyejin Shin, Sungwook Kim, Junbum Shin, and Xiaokui Xiao. 2018. Privacy enhanced matrix factorization for recommendation with local differential privacy. *IEEE Trans. Knowl. Data Eng.* 30, 9 (2018), 1770–1782.
- [115] Anu Shrestha, Francesca Spezzano, and Maria Soledad Pera. 2021. An empirical analysis of collaborative recommender systems robustness to shilling attacks. In *CEUR Workshop 3012* (2021), 45–57.
- [116] David Shriver, Sebastian Elbaum, Matthew B. Dwyer, and David S. Rosenblum. 2019. Evaluating recommender system stability with influence-guided fuzzing. In *AAAI*, Vol. 33. 4934–4942.
- [117] Brent Smith and Greg Linden. 2017. Two decades of recommender systems at Amazon.com. *IEEE Internet Comput.* 21, 3 (2017), 12–18.
- [118] Florian Strub, Jeremie Mary, and Preux Philippe. 2015. Collaborative filtering with stacked denoising autoencoders and sparse inputs. In *NeurIPS workshop on machine learning for eCommerce*.
- [119] Youchen Sun, Zhu Sun, Yingpeng Du, Jie Zhang, and Yew Soon Ong. 2024. Self-Supervised denoising through independent cascade graph augmentation for robust social recommendation. In *SIGKDD*. 2806–2817.
- [120] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. MIT Press.
- [121] Jinhui Tang, Xiaoyu Du, Xiangnan He, Fajie Yuan, Qi Tian, and Tat-Seng Chua. 2019. Adversarial training towards robust multimedia recommender system. *IEEE Trans. Knowl. Data Eng.* 32, 5 (2019), 855–867.
- [122] Jiaxi Tang, Hongyi Wen, and Ke Wang. 2020. Revisiting adversarially learned injection attacks against recommender systems. In *RecSys*. 318–327.
- [123] John K. Tarus, Zhendong Niu, and Ghulam Mustafa. 2018. Knowledge-based recommendation: A review of ontology-based recommender systems for e-learning. *Artif. Intell. Rev.* 50, 1 (2018), 21–48.
- [124] OpenAI Team. 2022. ChatGPT: Optimizing language models for dialogue. Retrieved August, 2023 from <https://openai.com/blog/>
- [125] Changxin Tian, Yuexiang Xie, Yaliang Li, Nan Yang, and Wayne Xin Zhao. 2022. Learning to denoise unreliable interactions for graph collaborative filtering. In *SIGIR*. 122–132.
- [126] Zhiyi Tian, Lei Cui, Jie Liang, and Shui Yu. 2022. A comprehensive survey on poisoning attacks and countermeasures in machine learning. *ACM Comput. Surv.* 55, 8 (2022), 1–35.
- [127] Raciél Yera Toledo, Yailé Caballero Mota, and Luis Martínez. 2015. Correcting noisy ratings in collaborative recommender systems. *Knowl.-Based Syst.* 76 (2015), 96–108.

- [128] Richard Tomsett, Amy Widdicombe, Tianwei Xing, Supriyo Chakraborty, Simon Julier, Prudhvi Gurram, Raghuveer M. Rao, and Mani B. Srivastava. 2018. Why the failure? How adversarial examples can provide insights for interpretable machine learning. In *FUSION*. IEEE, 838–845.
- [129] Thanh Tran, Renee Sweeney, and Kyumin Lee. 2019. Adversarial Mahalanobis distance-based attentive song recommender for automatic playlist continuation. In *SIGIR*. 245–254.
- [130] Dimitris Tsipras, Shibani Santurkar, Logan Engstrom, Alexander Turner, and Aleksander Madry. 2018. Robustness may be at odds with accuracy. In *ICLR*.
- [131] Joao Vinagre, Alipio Mário Jorge, Conceição Rocha, and Joao Gama. 2019. Statistically robust evaluation of stream-based recommender systems. *IEEE Trans. Knowl. Data Eng.* 33, 7 (2019), 2971–2982.
- [132] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *ICML*. 1096–1103.
- [133] Changsheng Wang, Jianbai Ye, Wenjie Wang, Chongming Gao, Fuli Feng, and Xiangnan He. 2023. Recad: Towards a unified library for recommender attack and defense. In *RecSys*. 234–244.
- [134] Haotao Wang, Tianlong Chen, Shupeng Gui, TingKui Hu, Ji Liu, and Zhangyang Wang. 2020. Once-for-all adversarial training: In-situ tradeoff between robustness and accuracy for free. In *NeurIPS*, Vol. 33. 7449–7461.
- [135] Hao Wang, Yao Xu, Cheng Yang, Chuan Shi, Xin Li, Ning Guo, and Zhiyuan Liu. 2023. Knowledge-adaptive contrastive learning for recommendation. In *WSDM*. 535–543.
- [136] Tianle Wang, Lianghao Xia, and Chao Huang. 2023. Denoised self-augmented learning for social recommendation. *arXiv preprint arXiv:2305.12685* (2023).
- [137] Wenjie Wang, Fuli Feng, Xiangnan He, Liqiang Nie, and Tat-Seng Chua. 2021. Denoising implicit feedback for recommendation. In *WSDM*. 373–381.
- [138] Xuezhi Wang, Haohan Wang, and Diyi Yang. 2022. Measure and improve robustness in NLP models: A survey. In *NAACL. ACL*, 4569–4586.
- [139] Yifan Wang, Weizhi Ma, Min Zhang, Yiqun Liu, and Shaoping Ma. 2023. A survey on the fairness of recommender systems. *ACM Trans. Inf. Syst.* 41, 3 (2023), 1–43.
- [140] Yu Wang, Xin Xin, Zaiqiao Meng, Joemon M. Jose, Fuli Feng, and Xiangnan He. 2022. Learning robust recommenders through cross-model agreement. In *TheWebConf*. 2015–2025.
- [141] Zongwei Wang, Min Gao, Wentao Li, Junliang Yu, Linxin Guo, and Hongzhi Yin. 2023. Efficient bi-level optimization for recommendation denoising. In *SIGKDD*. 2502–2511.
- [142] Zitai Wang, Qianqian Xu, Zhiyong Yang, Xiaochun Cao, and Qingming Huang. 2021. Implicit feedbacks are not always favorable: Iterative relabeled one-class collaborative filtering against noisy interactions. In *MM*. 3070–3078.
- [143] Chenwang Wu, Defu Lian, Yong Ge, Zhihao Zhu, Enhong Chen, and Senchao Yuan. 2021. Fight fire with fire: Towards robust recommender systems via adversarial poisoning training. In *SIGIR*. 1074–1083.
- [144] Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, et al. 2020. Mind: A large-scale dataset for news recommendation. In *ACL*. 3597–3606.
- [145] Jiancan Wu, Xiang Wang, Fuli Feng, Xiangnan He, Liang Chen, Jianxun Lian, and Xing Xie. 2021. Self-supervised graph learning for recommendation. In *SIGIR*. 726–735.
- [146] Shiwen Wu, Fei Sun, Wentao Zhang, Xu Xie, and Bin Cui. 2022. Graph neural networks in recommender systems: A survey. *ACM Comput. Surv.* 55, 5 (2022), 1–37.
- [147] Yao Wu, Christopher DuBois, Alice X. Zheng, and Martin Ester. 2016. Collaborative denoising auto-encoders for top-n recommender systems. In *WSDM*. 153–162.
- [148] Zhiang Wu, Junjie Wu, Jie Cao, and Dacheng Tao. 2012. HySAD: A semi-supervised hybrid shilling attack detector for trustworthy product recommendation. In *SIGKDD*. 985–993.
- [149] Jiafeng Xia, Dongsheng Li, Hansu Gu, Tun Lu, Peng Zhang, Li Shang, and Ning Gu. 2024. Neural Kalman filtering for robust temporal recommendation. In *WSDM*. 836–845.
- [150] Sicong Xie, Qunwei Li, Weidi Xu, Kaiming Shen, Shaohu Chen, and Wenliang Zhong. 2022. Denoising time cycle modeling for recommendation. In *SIGIR*. 1950–1955.
- [151] Xu Xie, Fei Sun, Zhaoyang Liu, Shiwen Wu, Jinyang Gao, Jiandong Zhang, Bolin Ding, and Bin Cui. 2022. Contrastive learning for sequential recommendation. In *ICDE*. IEEE, 1259–1273.
- [152] Da Xu, Chuanwei Ruan, Evren Korpeoglu, Sushant Kumar, and Kannan Achan. 2019. Self-attention with functional time representation learning. In *NeurIPS* 32 (2019).
- [153] Da Xu, Tobias Schnabel, Xiquan Cui, Sarah Dean, Aniket Deshmukh, Bo Yang, and Shipeng Yu. 2023. Foreword for workshop on decision making for Inf. Retr. and recommender systems. In *TheWebConf*. 920–920.
- [154] Jianpeng Xu, Lingfei Wu, Xiaolin Pang, Mohit Sharma, Dawei Yin, George Karypis, Justin Basilico, and Philip S. Yu. 2021. 2nd international workshop on industrial recommendation systems (IRS). In *SIGKDD*. 4173–4174.
- [155] Xiwang Yang, Yang Guo, Yong Liu, and Harald Steck. 2014. A survey of collaborative filtering based social recommender systems. *Comput. Commun.* 41 (2014), 1–10.

- [156] Yuhao Yang, Chao Huang, Lianghao Xia, and Chenliang Li. 2022. Knowledge graph contrastive learning for recommendation. In *SIGIR*. 1434–1443.
- [157] Zhihai Yang, Zhongmin Cai, and Yuan Yang. 2017. Spotting anomalous ratings for rating systems by analyzing target users and items. *Neurocomputing* 240 (2017), 25–46.
- [158] Zhihai Yang, Lin Xu, Zhongmin Cai, and Zongben Xu. 2016. Re-scale AdaBoost for attack detection in collaborative filtering recommender systems. *Knowl.-Based Syst.* 100 (2016), 74–88.
- [159] Haibo Ye, Xinjie Li, Yuan Yao, and Hanghang Tong. 2023. Towards robust neural graph collaborative filtering via structure denoising and embedding perturbation. *ACM Trans. Inf. Syst.* 41, 3 (2023), 1–28.
- [160] Raciél Yera, Jorge Castro, and Luis Martínez. 2016. A fuzzy model for managing natural noise in recommender systems. *Appl. Soft Comput.* 40 (2016), 187–198.
- [161] Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Jundong Li, and Zi Huang. 2024. Self-supervised learning for recommender systems: A survey. *IEEE TKDE* 36, 1 (2024), 335–355.
- [162] Yaodong Yu, Zitong Yang, Edgar Dobriban, Jacob Steinhardt, and Yi Ma. 2021. Understanding generalization in adversarial training via the bias-variance decomposition. *arXiv preprint arXiv:2103.09947* (2021).
- [163] Feng Yuan, Lina Yao, and Boualem Benatallah. 2019. Adversarial collaborative auto-encoder for top-n recommendation. In *IJCNN*. IEEE, 1–8.
- [164] Feng Yuan, Lina Yao, and Boualem Benatallah. 2019. Adversarial collaborative neural network for robust recommendation. In *SIGIR*. 1065–1068.
- [165] Feng Yuan, Lina Yao, and Boualem Benatallah. 2020. Exploring missing interactions: A convolutional generative adversarial network for collaborative filtering. In *CIKM*. 1773–1782.
- [166] Xiaoyong Yuan, Pan He, Qile Zhu, and Xiaolin Li. 2019. Adversarial examples: Attacks and defenses for deep learning. *IEEE Trans. Neural Netw. Learn. Syst.* 30, 9 (2019), 2805–2824.
- [167] Zhenrui Yue, Huimin Zeng, Ziyi Kou, Lanyu Shang, and Dong Wang. 2022. Defending substitution-based profile pollution attacks on sequential recommenders. In *RecSys*. 59–70.
- [168] Eva Zangerle and Christine Bauer. 2022. Evaluating recommender systems: Survey and framework. *ACM Comput. Surv.* 55, 8 (2022), 1–38.
- [169] Fuzhi Zhang, Yuanli Lu, Jianmin Chen, Shaoshuai Liu, and Zhoujun Ling. 2017. Robust collaborative filtering based on non-negative matrix factorization and R1-norm. *Knowl.-Based Syst.* 118 (2017), 177–190.
- [170] Fuzhi Zhang and Quanqiang Zhou. 2012. A meta-learning-based approach for detecting profile injection attacks in collaborative recommender systems. *J. Comput.* 7, 1 (2012), 226–234.
- [171] Fuzhi Zhang and Quanqiang Zhou. 2014. HHT-SVM: An online method for detecting profile injection attacks in collaborative recommender systems. *Knowl.-Based Syst.* 65 (2014), 96–105.
- [172] Kaike Zhang, Qi Cao, Gaolin Fang, Bingbing Xu, Hongjian Zou, Huawei Shen, and Xueqi Cheng. 2023. DyTed: Disentangled representation learning for discrete-time dynamic graph. In *SIGKDD*. 3309–3320.
- [173] Kaike Zhang, Qi Cao, Yunfan Wu, Fei Sun, Huawei Shen, and Xueqi Cheng. 2024. Improving the shortest plank: Vulnerability-aware adversarial training for robust recommender system. In *RecSys*. 680–689.
- [174] Kaike Zhang, Qi Cao, Yunfan Wu, Fei Sun, Huawei Shen, and Xueqi Cheng. 2024. Lorec: Combating poisons with large language model for robust sequential recommendation. In *SIGIR*. 1733–1742.
- [175] Kaike Zhang, Qi Cao, Yunfan Wu, Fei Sun, Huawei Shen, and Xueqi Cheng. 2024. Lorec: Large language model for robust sequential recommendation against poisoning attacks. *arXiv preprint arXiv:2401.17723* (2024).
- [176] Kaike Zhang, Qi Cao, Yunfan Wu, Fei Sun, Huawei Shen, and Xueqi Cheng. 2024. Understanding and improving adversarial collaborative filtering for robust recommendation. In *NeurIPS*.
- [177] Kaike Zhang, Qi Cao, Yunfan Wu, Fei Sun, Huawei Shen, and Xueqi Cheng. 2025. Personalized denoising implicit feedback for robust recommender system. In *TheWebConf*.
- [178] Kaike Zhang, Yunfan Wu, Yougang Lyu, Du Su, Yingqiang Ge, Shuchang Liu, Qi Cao, Zhaochun Ren, and Fei Sun. 2025. The 1st workshop on human-centered recommender systems. In *TheWebConf*.
- [179] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM Comput. Surv.* 52, 1 (2019), 1–38.
- [180] Shijie Zhang, Hongzhi Yin, Tong Chen, Quoc Viet Nguyen Hung, Zi Huang, and Lizhen Cui. 2020. GCN-based user representation learning for unifying robust recommendation and fraudster detection. In *SIGIR*. 689–698.
- [181] Wei Zhang, Junbing Yan, Zhuo Wang, and Jianyong Wang. 2022. Neuro-symbolic interpretable collaborative filtering for attribute-based recommendation. In *TheWebConf*. 3229–3238.
- [182] Yongfeng Zhang and Xu Chen. 2020. Explainable recommendation: A survey and new perspectives. *Found. Trends Inf. Retr.* 14, 1 (2020), 1–101.
- [183] Yongfeng Zhang, Xu Chen, Yi Zhang, and Xianjie Chen. 2021. CSR 2021: The 1st international workshop on causality in search and recommendation. In *SIGIR*. 2677–2680.

- [184] Yongfeng Zhang, Yunzhi Tan, Min Zhang, Yiqun Liu, Tat-Seng Chua, and Shaoping Ma. 2015. Catch the black sheep: Unified framework for shilling attack detection based on fraudulent action propagation. In *IJCAI*.
- [185] Yan Zhang, Ivor W. Tsang, Hongzhi Yin, Guowu Yang, Defu Lian, and Jingjing Li. 2020. Deep pairwise hashing for cold-start recommendation. *IEEE Trans. Knowl. Data Eng.* 34, 7 (2020), 3169–3181.
- [186] Zixing Zhang, Jürgen Geiger, Jouni Pohjalainen, Amr El-Desoky Mousa, Wenyu Jin, and Björn Schuller. 2018. Deep learning for environmentally robust speech recognition: An overview of recent developments. *ACM Trans. Intell. Syst. Technol.* 9, 5 (2018), 1–28.
- [187] Zhuo Zhang and Sanjeev R. Kulkarni. 2014. Detection of shilling attacks in recommender systems via spectral clustering. In *FUSION*. IEEE, 1–8.
- [188] Wayne Xin Zhao, Yupeng Hou, Xingyu Pan, Chen Yang, Zeyu Zhang, Zihan Lin, Jingsen Zhang, Shuqing Bian, Jiakai Tang, Wenqi Sun, et al. 2022. Recbole 2.0: Towards a more up-to-date recommendation library. In *CIKM*. 4722–4726.
- [189] Jiawei Zheng, Qianli Ma, Hao Gu, and Zhenjing Zheng. 2021. Multi-view denoising graph auto-encoders on heterogeneous information networks for cold-start recommendation. In *SIGKDD*. 2338–2348.
- [190] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *CIKM*. 1893–1902.
- [191] Wei Zhou, Yun Sing Koh, Junhao Wen, Shafiq Alam, and Gillian Dobbie. 2014. Detection of abnormal profiles on group attacks in recommender systems. In *SIGIR*. 955–958.
- [192] Wei Zhou, Junhao Wen, Yun Sing Koh, Qingyu Xiong, Min Gao, Gillian Dobbie, and Shafiq Alam. 2015. Shilling attacks detection in recommender systems based on target item analysis. *PloS One* 10, 7 (2015), e0130968.
- [193] Wei Zhou, Junhao Wen, Qingyu Xiong, Min Gao, and Jun Zeng. 2016. SVM-TIA a shilling attack detection method based on SVM and target item analysis in recommender systems. *Neurocomputing* 210 (2016), 197–205.
- [194] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. 2020. Deep graph contrastive representation learning. In *ICML Workshop on Graph Representation Learning*.
- [195] Jun Zou and Faramarz Fekri. 2013. A belief propagation approach for detecting shilling attacks in collaborative filtering. In *CIKM*. 1837–1840.

A APPENDIX

A.1 Evaluation with Certifiable Robustness

For simplicity and without loss of generality, we assume $f(\mathbf{x}) \leq 0$. Consequently, in subsequent discussions, we solely show the process of maximizing the value of δ . In the non-robust certification approach, a greedy strategy is utilized to identify the dimension j of $\mathbf{x}$ that maximizes the impact on the output. This dimension is given by

$$j^* = \arg \max_j w_j + \sum_{f=1}^k \sum_{i=1}^d v_{i,f} v_{j,f} x_i + \sum_{f=1}^k v_{i,f}^2 \quad (30)$$

To compute $\mathbf{x}'$, they iteratively flip the j dimension of $\mathbf{x}$ until the total number of flipped dimensions is equal to budget p . If the prediction of the perturbed $\mathbf{x}'$ differs from that of $\mathbf{x}$, i.e., $\text{sign}(f(\mathbf{x})) \neq \text{sign}(f(\mathbf{x}'))$, the model f is considered certifiably non-robust under the budget q for the instance $\mathbf{x}$. It's worth noting that the calculation of δ in the non-robust certification through the greedy strategy might not yield the maximum value. Therefore, even if we cannot identify a perturbed $\mathbf{x}'$ with a prediction different from that of $\mathbf{x}$, we cannot definitively assert the model's f robustness.

In contrast, robust certification seeks to ascertain the model's f robustness by approximating the upper bound $\delta_{\max}$ of the prediction change under the perturbation budget q . Liu et al. [83] split δ into two problems, i.e., $\delta = \delta_1(\mathbf{x}) + \delta_2(\mathbf{x})$, that are easy to solve the upper bound $\delta_{\max}$:

$$\delta_1(\mathbf{x}) = \sum_{j=1}^d w_j x'_j + \sum_{f=1}^k \sum_{i=1}^d \sum_{j=1}^d v_{i,f} v_{j,f} x_i x'_j, \quad \delta_2(\mathbf{x}) = \frac{1}{2} \sum_{f=1}^k \left(\sum_{j=1}^d v_{j,f} x'_j \right)^2 + \frac{1}{2} \sum_{f=1}^k \sum_{j=1}^d v_{j,f}^2 x_j'^2, \quad (31)$$

Table 7. Nomenclature

Abbreviation	Description
$(u, i, r_{u,i})$	User-item-rating triplets
$\mathcal{U}$	User set, $u \in \mathcal{U}$
$\mathcal{I}$	Item set, $i \in \mathcal{I}$
$\mathcal{R}$	Interaction set, $r_{u,i} \in \mathcal{R}$
$\mathcal{I}_u$	The items related to user u
$\mathcal{R}_u / \mathcal{R}_i$	The ratings given by user u / The ratings related to item i
$\mathcal{D}$	Collected dataset, $(u, i, r_{u,i}) \in \mathcal{D}$
$\mathcal{D}^+$	Training dataset
$\mathcal{D}^-$	Test dataset
Δ	Perturbations of data
f_*	recommender systems trained on $*$
M / M'	Performance of recommender systems trained on clean data / perturbed data
$\psi(\cdot, \cdot)$	Error function
$\delta(\cdot, \cdot)$	Difference function
$\mathcal{L}$	Loss function
Θ	Model parameters
L_u	Ground truth preferred list for user u
$\hat{L}_u@k / \hat{L}'_u@k$	Top-k recommendation list for user u given by recommender systems trained on clean data / perturbed data

The approximated upper bound $\bar{\delta}$ can be computed by $\bar{\delta} = \max_{\mathbf{x}} \delta_1(\mathbf{x}) + \delta_2(\mathbf{x})$, where $\bar{\delta} \geq \delta_{\max}$. If the approximated upper bound $\bar{\delta}$ does not alter the prediction, i.e., $\text{sign}(f(\mathbf{x})) = \text{sign}(f(\mathbf{x}) + \bar{\delta})$, the model f can be declared certifiably robust. However, if the upper bound $\bar{\delta}$ does change the prediction, we cannot claim that the model f is non-robust because $\bar{\delta} \geq \delta_{\max}$.

A.2 Datasets

MovieLens.⁹ GroupLens Research has collected and provided rating datasets from the MovieLens website. These datasets come in different sizes, with the most commonly used versions being MovieLens-100K, MovieLens-1M, and MovieLens-20M. MovieLens-100K was collected over a seven-month period (19 September 1997 – 22 April 1998) and has been cleaned by removing users with fewer than 20 ratings or incomplete demographic information (age, gender, occupation, zip code). MovieLens-1M was collected in 2000, comprising users who joined MovieLens during that year. MovieLens-20M (9 January 1995 – 31 March 2015) includes randomly selected users who had rated at least 20 movies but does not contain demographic information.

Amazon.¹⁰ This dataset encompasses Amazon reviews (ratings, text, helpfulness votes), product metadata (descriptions, category information, price, brand, and image features), and links (also viewed/also bought graphs). It consists of 29 subcategories, such as Beauty, Books, Movies, and Electronics. The most frequently used subsets are Amazon-Beauty and Amazon-Book. Additionally, each dataset has meta, rating-only, and k-score versions. The meta version contains all available information, including reviews, product metadata, and links. The rating-only version includes user ratings for items, while the k-score is a dense subset where each remaining user and item has at least k reviews.

⁹<https://grouplens.org/datasets/movielens/>

¹⁰<https://nijianmo.github.io/amazon/index.html>

LastFM.¹¹ LastFM 1K is a dataset released by LastFM, collecting the entire listening history (approximately 20 million records) of 992 users from July 2005 to May 2009. Additionally, it includes user information such as gender, age, country, and registration time.

Netflix.¹² The Netflix movie rating dataset contains over 100 million ratings from 480 thousand randomly chosen, anonymous Netflix customers across 17 thousand movie titles. The data, collected between October 1998 and December 2005, reflects the distribution of all ratings received during this period. Ratings are on a scale from 1 to 5 (integral) stars.

Yelp.¹³ The Yelp dataset is a subset of Yelp businesses and reviews, featuring over 1.2 million business attributes such as hours, parking availability, and ambiance. It covers 11 metropolitan areas.

MIND.¹⁴ The MIND dataset is a large-scale dataset for news recommendation research. It is collected from anonymized behavior logs of the Microsoft News website.

Received 5 September 2023; revised 31 March 2025; accepted 12 July 2025

¹¹<http://ocelma.net/MusicRecommendationDataset/lastfm-1K.html>

¹²<https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data>

¹³<https://www.yelp.com/dataset>

¹⁴<https://msnews.github.io/>